Assignment for Langflow

Build a test case reviewer using Langflow. Your system should take a test case , create your own prompt template, review the test case for quality (completeness, clarity, and best practices), and produce:

List of issues

Suggestions for improvement

An improved version of the test case

Input Test cases :

Test Cases for Doctor Prescription Verification Workflow

Test Case 1: Display Pending Prescriptions

- **Test Case ID:** TC_001
- **Title:** Verify system displays all pending prescriptions for verification
- **Module:** Prescription Verification
- **Priority:** High
- **Preconditions:** Doctor logged in, patient record accessible
- **Test Steps:**
  1. Log in as a doctor.
  2. Access a patient's record.
  3. Open the prescription module.
- **Test Data:** Patient with pending prescriptions
- **Expected Result:** All pending prescriptions are displayed for verification.

Test Case 2: Highlight Dosage Conflicts

- **Test Case ID:** TC_002

- **Title:** Verify system highlights dosage conflicts
- **Module:** Prescription Verification
- **Priority:** High
- **Preconditions:** Doctor logged in, prescription with dosage conflict
- **Test Steps:**
  1. Log in as a doctor.
  2. Access a patient's record with a prescription containing a dosage conflict.
  3. Review the prescription.
- **Test Data:** Prescription with a known dosage conflict
- **Expected Result:** System highlights the dosage conflict and provides a warning message.

Test Case 3: Detect Duplicate Medicines

- **Test Case ID:** TC_003
- **Title:** Verify system detects duplicate medicines
- **Module:** Prescription Verification
- **Priority:** High
- **Preconditions:** Doctor logged in, prescription with duplicate medicines
- **Test Steps:**
  1. Log in as a doctor.
  2. Access a patient's record with a prescription containing duplicate medicines.
  3. Review the medication list.
- **Test Data:** Prescription with duplicate medicines listed
- **Expected Result:** System highlights duplicate medicines and displays a warning message.

Test Case 4: AI-Based Recommendations

- **Test Case ID:** TC_004

- **Title:** Verify system provides AI-based recommendations for corrections

- **Module:** AI Validation

- **Priority:** High

- **Preconditions:** Doctor logged in, prescription with errors

- **Test Steps:**

  1. Log in as a doctor.

  2. Access a patient's record with a prescription containing errors.

  3. Review the prescription and AI recommendations.

- **Test Data:** Prescription with known errors (dosage conflict or duplicate medicines)

- **Expected Result:** System provides AI-based recommendations for correcting the errors.

Test Case 5: Approve Prescription

- **Test Case ID:** TC_005

- **Title:** Verify approved prescription syncs with pharmacy queue

- **Module:** Pharmacy Integration

- **Priority:** High

- **Preconditions:** Doctor logged in, verified prescription

- **Test Steps:**

  1. Log in as a doctor.

  2. Access a verified prescription.

  3. Approve the prescription.

- **Test Data:** Verified prescription

- **Expected Result:** Prescription is automatically updated in the pharmacy dispensing queue.

Test Case 6: Negative Scenario - Invalid Login

- **Test Case ID:** TC_006
- **Title:** Verify system response to invalid doctor login
- **Module:** UI
- **Priority:** Medium
- **Preconditions:** Invalid doctor credentials
- **Test Steps:**
  1. Attempt to log in with invalid credentials.
- **Test Data:** Invalid username or password
- **Expected Result:** System denies access and displays an error message.

Test Case 7: Edge Case - Empty Prescription List

- **Test Case ID:** TC_007
- **Title:** Verify system response to an empty prescription list
- **Module:** Prescription Verification
- **Priority:** Medium
- **Preconditions:** Doctor logged in, patient with no prescriptions
- **Test Steps:**
  1. Log in as a doctor.
  2. Access a patient's record with no prescriptions.
  3. Open the prescription module.
- **Test Data:** Patient with no prescriptions
- **Expected Result:** System displays a message indicating no prescriptions are available for verification.

Test Case 8: Boundary Condition - Maximum Dosage

- **Test Case ID:** TC_008
- **Title:** Verify system response to maximum dosage
- **Module:** Prescription Verification
- **Priority:** Medium
- **Preconditions:** Doctor logged in, prescription with maximum dosage
- **Test Steps:**
  1. Log in as a doctor.
  2. Access a patient's record with a prescription at the maximum dosage.
  3. Review the prescription.
- **Test Data:** Prescription with maximum allowable dosage
- **Expected Result:** System does not highlight the dosage as an error but may provide information on the maximum dosage.

Test Case 9: Error Handling - Network Failure

- **Test Case ID:** TC_009
- **Title:** Verify system response to network failure during prescription sync
- **Module:** Pharmacy Integration
- **Priority:** High
- **Preconditions:** Doctor logged in, verified prescription, simulated network failure
- **Test Steps:**
  1. Log in as a doctor.
  2. Access a verified prescription.
  3. Simulate a network failure.
  4. Attempt to approve the prescription.
- **Test Data:** Verified prescription, network failure simulation

- **Expected Result:** System displays an error message indicating the failure to sync with the pharmacy queue due to network issues and provides an option to retry.

Test Case 10: Validation Scenario - Drug Interaction

- **Test Case ID:** TC_010

- **Title:** Verify system detects drug interactions

- **Module:** Prescription Verification

- **Priority:** High

- **Preconditions:** Doctor logged in, prescription with potential drug interactions

- **Test Steps:**

  1. Log in as a doctor.

  2. Access a patient's record with a prescription containing potential drug interactions.

  3. Review the prescription.

- **Test Data:** Prescription with known drug interactions

- **Expected Result:** System highlights the potential drug interactions and provides a warning message.

Prompt:

I - Instruction:

You are a Senior QA Automation Architect. Your task is to review the given test case thoroughly and evaluate its quality based on completeness, clarity, correctness, and adherence to testing best practices.

C - Context:

The test cases include:

- Test Case ID

- Title

- Preconditions

- Test Steps

- Test Data

- Expected Results

- Actual Results (optional)

- Priority/Severity (optional)

You need to identify missing elements, unclear steps, poor structure, and non-testable conditions. Assume this test case will be used in a production-level QA process.

E - Examples:(Reference Format Only)

Test Case 1: Display Pending Prescriptions

- **Test Case ID:** TC_001

- **Title:** Verify system displays all pending prescriptions for verification

- **Module:** Prescription Verification

- **Priority:** High

- **Test Data:** Patient with pending prescriptions

- **Expected Result:** All pending prescriptions are displayed for verification.

(Output should identify issues and improvements)

Improved Version:(Reference Format Only)

Test Case 1: Display Pending Prescriptions

- **Test Case ID:** TC_001

- **Title:** Verify system displays all pending prescriptions for verification

- **Module:** Prescription Verification

- **Priority:** High

- - **Test Steps:**

  1. Log in as a doctor.

  2. Access a patient's record.

  3. Open the prescription module.

- **Test Data:** Patient with pending prescriptions

- **Expected Result:** All pending prescriptions are displayed for verification.

P - Persona:

Act as a detail-oriented QA expert with strong knowledge in manual testing, test design techniques, and automation readiness. Be critical but constructive.

O - Output Format:

Provide the response strictly in the following structure:

1. Issues:

- Issue 1:

- Issue 2:

- Issue 3:

2. Suggestions for Improvement:

- Suggestion 1

- Suggestion 2

- Suggestion 3

3. Improved Test Case:

- Well-structured and formatted test case

- Include all necessary fields (ID, Title, Preconditions, Steps, Expected Results, etc.)

- Make it clear, complete, and automation-friendly

T - Tone:

Professional, precise, and constructive. Avoid unnecessary verbosity. Focus on clarity and practical improvements.

{testcase}

Output :

1. Issues

- **Issue 1 – Incomplete Test Case Structure**

  Most cases omit several standard fields (e.g., *Module* is present but *Severity*, *Test Environment*, *Post-conditions*, *Attachments*, *Automation Status* are missing). This makes the cases unsuitable for traceability and for feeding into test-management tools.

- **Issue 2 – Ambiguous / Non-testable Preconditions & Test Data**

  Preconditions such as “Doctor logged in, patient record accessible” do not specify *how* the doctor is logged in (username/password, role), nor how the patient record is identified (patient ID, name). Test data descriptions (“Patient with pending prescriptions”) are vague; they do not define the exact data set, its source, or any required setup steps.

- **Issue 3 – Vague Test Steps & Expected Results**

  Steps are written at a high level (“Access a patient’s record”) without indicating the UI path, navigation elements, or verification points. Expected results are generic (“System highlights …”) and do not describe the exact UI element, message text, color, or location, making the test non-deterministic and hard to automate.

- **Issue 4 – Missing Validation of Negative Paths & Edge Cases**

Only one negative scenario (invalid login) is present. Other critical negative flows (e.g., “Approve prescription when AI recommendation is rejected”, “System behavior when duplicate detection fails”) are absent.

- **Issue 5 – No Traceability or Requirement Reference**

There is no link to functional/clinical requirements, user stories, or regulatory standards (e.g., FDA, HIPAA). This hampers impact analysis when requirements change.

- **Issue 6 – Lack of Automation-Readiness Details**

No identifiers for UI elements (e.g., element IDs, CSS selectors), no data-driven approach, and no indication of whether the test is manual-only or automated.

- **Issue 7 – Inconsistent Naming & ID Conventions**

Test IDs are all caps with an underscore (TC_001) – acceptable – but some titles repeat the module name, leading to redundancy. Also, the “Priority” field is used but “Severity” is missing, which is a best-practice gap.

- **Issue 8 – Missing Cleanup / Post-condition Steps**

After actions such as “Approve the prescription”, there is no step to revert the state (e.g., delete the test prescription) which can cause data pollution in the test environment.

---

2. Suggestions for Improvement

1. **Adopt a Full-Featured Test Case Template**

Include fields: Test Case ID, Title, Module, Requirement ID(s), Priority, Severity, Preconditions, Test Data (with source & exact values), Test Environment, Test Steps (with expected result per step), Post-conditions, Attachments, Automation Status, Owner, and Execution History.

2. **Make Preconditions & Test Data Explicit and Reproducible**

- Specify exact user credentials (e.g., `doctor_user = "dr.smith", password = "Pass!23"`).

- Provide patient identifiers (e.g., `PatientID = "P123456"`).

- Describe how the test data is seeded (SQL script, API call, UI setup) and include the script or API payload.

3. **Detail Each Step with Expected Result**

Use a “Step – Action – Expected Result” table. Include UI element identifiers, exact message text, color codes, and any system logs that should be verified. This makes the test deterministic and automation-friendly.

4. **Add Negative, Boundary, and Cleanup Steps**

For every positive flow, add at least one negative verification (e.g., attempt to approve without AI confirmation). Include a cleanup step that removes or resets test data.

5. **Link to Requirements & Regulatory Standards**

Add a “Requirement(s)” column (e.g., `REQ-RX-001 – Display pending prescriptions`). This enables traceability and impact analysis.

6. **Provide Automation Hooks**

Include element locators (e.g., `id="prescriptionList"`), API endpoints, and data-driven parameters. Mark the test as “Automatable – Yes/No” and note any manual-only steps.

7. **Standardize Naming & Prioritization**

Use consistent naming: `<Module> – <Feature> – <Condition>`. Add both Priority (business impact) and Severity (technical impact) fields.

---

3. Improved Test Case (TC_001 – Display Pending Prescriptions)

```

---

Test Case ID : TC_001

Title : Prescription Verification – Display Pending Prescriptions

Module : Prescription Verification

Requirement ID(s) : REQ-RX-001, REQ-RX-002

Priority : High

Severity : Critical

Test Type : Functional / UI

Automation Status : Automatable – Yes

Owner : QA Lead – Jane Doe

Created Date : 2026-04-30

Last Updated : 2026-04-30

---

Preconditions

1. Test environment: DEV-EHR-01 (URL: https://dev-ehr.example.com)
2. Database seeded with patient record:
   - PatientID = "P123456"
   - Name = "John Doe"
   - At least 2 pending prescriptions (PrescriptionIDs: PRX001, PRX002)
3. Doctor user exists:
   - Username = "dr.smith"
   - Password = "Pass!23"
   - Role = "Doctor"
4. No active session for dr.smith (ensure logout from previous runs)
```

Test Data

Field	Value
Doctor Username	dr.smith
Doctor Password	Pass!23
Patient ID	P123456
Expected Presc.	PRX001, PRX002

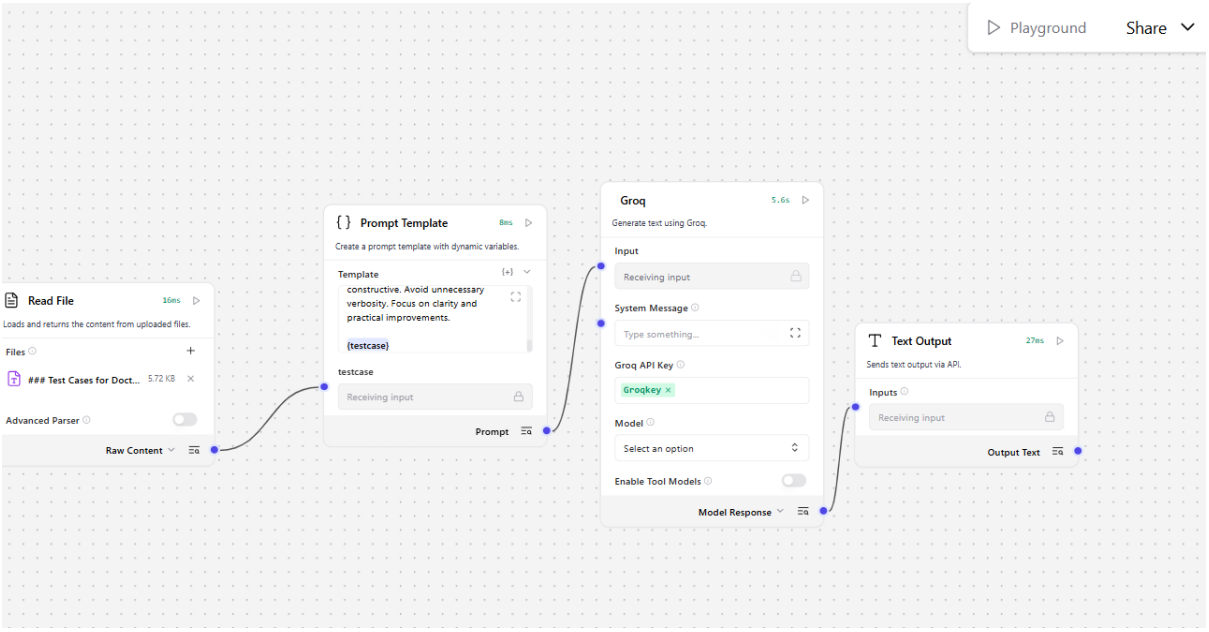
Test Environment

Browser : Chrome 124.0.6367.91 (headless for automation)

OS : Windows 11

Network : Stable (latency < 50 ms)

Screenshot:



Exported Json file data:


```
{
  "data": {
    "edges": [
      {
        "animated": false,
        "className": "",
        "data": {
          "sourceHandle": {
            "dataType": "Prompt Template",
            "id": "Prompt Template-TU3sw",
            "name": "prompt",
            "output_types": [
              "Message"
            ]
          },
          "targetHandle": {
            "fieldName": "input_value",
            "id": "GroqModel-YiI97",
            "inputTypes": [
              "Message"
            ],
            "type": "str"
          }
        },
        "id": "xy-edge__Prompt Template-TU3sw{œdataTypeœ:œPrompt Templateœ,œidœ:œPrompt Template-TU3swœ,œnameœ:œpromptœ,œoutput_typesœ:[œMessageœ]}-GroqModel-
```

YiI97{œfieldNameœ:œinput_valueœ,œidœ:œGroqModel-YiI97œ,œinputTypesœ:[œMessageœ],œtypeœ:œstrœ}",

"selected": false,

"source": "Prompt Template-TU3sw",

"sourceHandle": "{œdataTypeœ:œPrompt Templateœ,œidœ:œPrompt Template-TU3swœ,œnameœ:œpromptœ,œoutput_typesœ:[œMessageœ]}",

"target": "GroqModel-YiI97",

"targetHandle": "{œfieldNameœ:œinput_valueœ,œidœ:œGroqModel-YiI97œ,œinputTypesœ:[œMessageœ],œtypeœ:œstrœ}"

},

{

"animated": false,

"className": "",

"data": {

"sourceHandle": {

"dataType": "File",

"id": "File-uI9B7",

"name": "message",

"output_types": [

"Message"

]

},

"targetHandle": {

"fieldName": "testcase",

"id": "Prompt Template-TU3sw",

"inputTypes": [

"Message"

],

"type": "str"

```
      }
    },
    "id": "xy-edge__File-uI9B7{œdataTypeœ:œFileœ,œidœ:œFile-uI9B7œ,œnameœ:œmessageœ,œoutput_typesœ:[œMessageœ]}-Prompt Template-TU3sw{œfieldNameœ:œtestcaseœ,œidœ:œPrompt Template-TU3swœ,œinputTypesœ:[œMessageœ],œtypeœ:œstrœ}",
    "selected": false,
    "source": "File-uI9B7",
    "sourceHandle": "{œdataTypeœ:œFileœ,œidœ:œFile-uI9B7œ,œnameœ:œmessageœ,œoutput_typesœ:[œMessageœ]}",
    "target": "Prompt Template-TU3sw",
    "targetHandle": "{œfieldNameœ:œtestcaseœ,œidœ:œPrompt Template-TU3swœ,œinputTypesœ:[œMessageœ],œtypeœ:œstrœ}"
  },
  {
    "animated": false,
    "className": "",
    "data": {
      "sourceHandle": {
        "dataType": "GroqModel",
        "id": "GroqModel-YiI97",
        "name": "text_output",
        "output_types": [
          "Message"
        ]
      },
      "targetHandle": {
        "fieldName": "input_value",
        "id": "TextOutput-gW8O8",
```

```
        "inputTypes": [
          "Message"
        ],
        "type": "str"
      }
    },
    "id": "xy-edge__GroqModel-YiI97{œdataTypeœ:œGroqModelœ,œidœ:œGroqModel-YiI97œ,œnameœ:œtext_outputœ,œoutput_typesœ:[œMessageœ]}-TextOutput-gW8O8{œfieldNameœ:œinput_valueœ,œidœ:œTextOutput-gW8O8œ,œinputTypesœ:[œMessageœ],œtypeœ:œstrœ}",
    "selected": false,
    "source": "GroqModel-YiI97",
    "sourceHandle": "{œdataTypeœ:œGroqModelœ,œidœ:œGroqModel-YiI97œ,œnameœ:œtext_outputœ,œoutput_typesœ:[œMessageœ]}",
    "target": "TextOutput-gW8O8",
    "targetHandle": "{œfieldNameœ:œinput_valueœ,œidœ:œTextOutput-gW8O8œ,œinputTypesœ:[œMessageœ],œtypeœ:œstrœ}"
  }
 ],
 "nodes": [
  {
   "data": {
    "id": "Prompt Template-TU3sw",
    "node": {
     "base_classes": [
      "Message"
     ],
     "beta": false,
     "conditional_paths": [],
```

```
"custom_fields": {
  "template": [
    "testcase"
  ]
},
"description": "Create a prompt template with dynamic variables.",
"display_name": "Prompt Template",
"documentation": "https://docs.langflow.org/components-prompts",
"edited": false,
"error": null,
"field_order": [
  "template",
  "use_double_brackets",
  "tool_placeholder"
],
"frozen": false,
"full_path": null,
"icon": "prompts",
"is_composition": null,
"is_input": null,
"is_output": null,
"legacy": false,
"lf_version": "1.8.2",
"metadata": {
  "code_hash": "5b3e6730923e",
  "dependencies": {
    "dependencies": [
      {
```

```
      "name": "lfx",
      "version": null
    }
   ],
   "total_dependencies": 1
  },
  "module": "lfx.components.models_and_agents.prompt.PromptComponent"
},
"minimized": false,
"name": "",
"output_types": [],
"outputs": [
 {
  "allows_loop": false,
  "cache": true,
  "display_name": "Prompt",
  "group_outputs": false,
  "hidden": null,
  "loop_types": null,
  "method": "build_prompt",
  "name": "prompt",
  "options": null,
  "required_inputs": null,
  "selected": "Message",
  "tool_mode": true,
  "types": [
   "Message"
  ],
```

```
      "value": "__UNDEFINED__"
     }
    ],
    "pinned": false,
    "priority": null,
    "replacement": null,
    "template": {
     "_type": "Component",
     "code": {
      "advanced": true,
      "dynamic": true,
      "fileTypes": [],
      "file_path": "",
      "info": "",
      "list": false,
      "load_from_db": false,
      "multiline": true,
      "name": "code",
      "password": false,
      "placeholder": "",
      "required": true,
      "show": true,
      "title_case": false,
      "type": "code",
      "value": "from typing import Any\n\nfrom lfx.base.prompts.api_utils import process_prompt_template\nfrom lfx.custom.custom_component.component import Component\nfrom lfx.inputs.input_mixin import FieldTypes\nfrom lfx.inputs.inputs import DefaultPromptField\nfrom lfx.io import BoolInput, MessageTextInput, Output, PromptInput\nfrom lfx.log.logger import logger\nfrom lfx.schema.dotdict import
```

dotdict\nfrom lfx.schema.message import Message\nfrom lfx.template.utils import update_template_values\nfrom lfx.utils.mustache_security import validate_mustache_template\n\n\nclass PromptComponent(Component):\n display_name: str = \"Prompt Template\"\n description: str = \"Create a prompt template with dynamic variables.\"\n documentation: str = \"https://docs.langflow.org/components-prompts\"\n icon = \"prompts\"\n trace_type = \"prompt\"\n name = \"Prompt Template\"\n\n inputs = [\n PromptInput(name=\"template\", display_name=\"Template\"),\n BoolInput(\n name=\"use_double_brackets\",\n display_name=\"Use Double Brackets\",\n value=False,\n advanced=True,\n info=\"Use {{variable}} syntax instead of {variable}.\",\n real_time_refresh=True,\n),\n MessageTextInput(\n name=\"tool_placeholder\",\n display_name=\"Tool Placeholder\",\n tool_mode=True,\n advanced=True,\n info=\"A placeholder input for tool mode.\",\n),\n]\n\n outputs = [\n Output(display_name=\"Prompt\", name=\"prompt\", method=\"build_prompt\"),\n]\n\n def update_build_config(self, build_config: dotdict, field_value: Any, field_name: str | None = None) -> dotdict:\n \"\"\"Update the template field type based on the selected mode.\"\"\"\n if field_name == \"use_double_brackets\":\n # Change the template field type based on mode\n is_mustache = field_value is True\n if is_mustache:\n build_config[\"template\"][\"type\"] = FieldTypes.MUSTACHE_PROMPT.value\n else:\n build_config[\"template\"][\"type\"] = FieldTypes.PROMPT.value\n\n # Re-process the template to update variables when mode changes\n template_value = build_config.get(\"template\", {}).get(\"value\", \"\")\n if template_value:\n # Ensure custom_fields is properly initialized\n if \"custom_fields\" not in build_config:\n build_config[\"custom_fields\"] = {}\n\n # Clean up fields from the OLD mode before processing with NEW mode\n # This ensures we don't keep fields with wrong syntax even if validation fails\n old_custom_fields = build_config[\"custom_fields\"].get(\"template\", [])\n for old_field in list(old_custom_fields):\n # Remove the field from custom_fields and template\n if old_field in old_custom_fields:\n old_custom_fields.remove(old_field)\n build_config.pop(old_field, None)\n\n # Try to process template with new mode to add new variables\n # If validation fails, at least we cleaned up old fields\n try:\n # Validate mustache templates for security\n if is_mustache:\n validate_mustache_template(template_value)\n\n # Re-process template with new mode to add new variables\n _ = process_prompt_template(\n template=template_value,\n name=\"template\",\n custom_fields=build_config[\"custom_fields\"],\n frontend_node_template=build_config,\n is_mustache=is_mustache,\n)\n except ValueError as e:\n # If validation fails, we still updated the mode and cleaned old fields\n # User will see error when they try to save\n

```
logger.debug(f\"Template validation failed during mode switch: {e}\")\n        return build_config\n\n    async def build_prompt(self) -> Message:\n        use_double_brackets = self.use_double_brackets if hasattr(self, \"use_double_brackets\") else False\n        template_format = \"mustache\" if use_double_brackets else \"f-string\"\n        prompt = await Message.from_template_and_variables(template_format=template_format, **self._attributes)\n        self.status = prompt.text\n        return prompt\n\n    def _update_template(self, frontend_node: dict):\n        prompt_template = frontend_node[\"template\"][\"template\"][\"value\"]\n        use_double_brackets = frontend_node[\"template\"].get(\"use_double_brackets\", {}).get(\"value\", False)\n        is_mustache = use_double_brackets is True\n\n        try:\n            # Validate mustache templates for security\n            if is_mustache:\n                validate_mustache_template(prompt_template)\n\n            custom_fields = frontend_node[\"custom_fields\"]\n            frontend_node_template = frontend_node[\"template\"]\n            _ = process_prompt_template(\n                template=prompt_template,\n                name=\"template\",\n                custom_fields=custom_fields,\n                frontend_node_template=frontend_node_template,\n                is_mustache=is_mustache,\n            )\n        except ValueError as e:\n            # If validation fails, don't add variables but allow component to be created\n            logger.debug(f\"Template validation failed in _update_template: {e}\")\n        return frontend_node\n\n    async def update_frontend_node(self, new_frontend_node: dict, current_frontend_node: dict):\n        \"\"\"This function is called after the code validation is done.\"\"\"\n        frontend_node = await super().update_frontend_node(new_frontend_node, current_frontend_node)\n        template = frontend_node[\"template\"][\"template\"][\"value\"]\n        use_double_brackets = frontend_node[\"template\"].get(\"use_double_brackets\", {}).get(\"value\", False)\n        is_mustache = use_double_brackets is True\n\n        try:\n            # Validate mustache templates for security\n            if is_mustache:\n                validate_mustache_template(template)\n\n            # Kept it duplicated for backwards compatibility\n            _ = process_prompt_template(\n                template=template,\n                name=\"template\",\n                custom_fields=frontend_node[\"custom_fields\"],\n                frontend_node_template=frontend_node[\"template\"],\n                is_mustache=is_mustache,\n            )\n        except ValueError as e:\n            # If validation fails, don't add variables but allow component to be updated\n            logger.debug(f\"Template validation failed in update_frontend_node: {e}\")\n        # Now that template is updated, we need to grab any values that were set in the current_frontend_node\n        # and update the frontend_node with those values\n        update_template_values(new_template=frontend_node, previous_template=current_frontend_node[\"template\"])\n        return frontend_node\n\n    def _get_fallback_input(self, **kwargs):\n        return DefaultPromptField(**kwargs)\n"
```

```
},
"template": {
 "_input_type": "PromptInput",
 "advanced": false,
 "display_name": "Template",
 "dynamic": false,
 "info": "",
 "list": false,
 "list_add_label": "Add More",
 "name": "template",
 "override_skip": false,
 "placeholder": "",
 "required": false,
 "show": true,
 "title_case": false,
 "tool_mode": false,
 "trace_as_input": true,
 "track_in_telemetry": false,
 "type": "prompt",
 "value": "I - Instruction:\nYou are a Senior QA Automation Architect. Your task is to review the given test case thoroughly and evaluate its quality based on completeness, clarity, correctness, and adherence to testing best practices.\n\nC - Context:\nThe test cases include:\n- Test Case ID\n- Title\n- Preconditions\n- Test Steps\n- Test Data\n- Expected Results\n- Actual Results (optional)\n- Priority/Severity (optional)\n\nYou need to identify missing elements, unclear steps, poor structure, and non-testable conditions. Assume this test case will be used in a production-level QA process.\n\nE - Examples:(Reference Format Only)\n  Test Case 1: Display Pending Prescriptions\n- **Test Case ID:** TC_001\n- **Title:** Verify system displays all pending prescriptions for verification\n- **Module:** Prescription Verification\n- **Priority:** High\n- **Test Data:** Patient with pending prescriptions\n- **Expected Result:** All pending prescriptions are displayed for verification.\n\n(Output should identify issues and improvements)\n\nImproved Version:(Reference Format Only)\nTest
```

Case 1: Display Pending Prescriptions\n- **Test Case ID:** TC_001\n- **Title:** Verify system displays all pending prescriptions for verification\n- **Module:** Prescription Verification\n- **Priority:** High\n- - **Test Steps:**\n 1. Log in as a doctor.\n 2. Access a patient's record.\n 3. Open the prescription module.\n- **Test Data:** Patient with pending prescriptions\n- **Expected Result:** All pending prescriptions are displayed for verification.\n\n\nP - Persona:\nAct as a detail-oriented QA expert with strong knowledge in manual testing, test design techniques, and automation readiness. Be critical but constructive.\n\nO - Output Format:\nProvide the response strictly in the following structure:\n\n1. Issues:\n- Issue 1:\n- Issue 2: \n- Issue 3:\n\n2. Suggestions for Improvement:\n- Suggestion 1\n- Suggestion 2\n- Suggestion 3\n\n3. Improved Test Case:\n- Well-structured and formatted test case\n- Include all necessary fields (ID, Title, Preconditions, Steps, Expected Results, etc.)\n- Make it clear, complete, and automation-friendly\n\nT - Tone:\nProfessional, precise, and constructive. Avoid unnecessary verbosity. Focus on clarity and practical improvements.\n\n{testcase}"

},

"testcase": {

"advanced": false,

"display_name": "testcase",

"dynamic": false,

"field_type": "str",

"fileTypes": [],

"file_path": "",

"info": "",

"input_types": [

"Message"

],

"list": false,

"load_from_db": false,

"multiline": true,

"name": "testcase",

"placeholder": "",

"required": false,

```
  "show": true,
  "title_case": false,
  "type": "str",
  "value": ""
 },
 "tool_placeholder": {
  "_input_type": "MessageTextInput",
  "advanced": true,
  "display_name": "Tool Placeholder",
  "dynamic": false,
  "info": "A placeholder input for tool mode.",
  "input_types": [
   "Message"
  ],
  "list": false,
  "list_add_label": "Add More",
  "load_from_db": false,
  "name": "tool_placeholder",
  "override_skip": false,
  "placeholder": "",
  "required": false,
  "show": true,
  "title_case": false,
  "tool_mode": true,
  "trace_as_input": true,
  "trace_as_metadata": true,
  "track_in_telemetry": false,
  "type": "str",
```

```
    "value": ""
   },
   "use_double_brackets": {
    "_input_type": "BoolInput",
    "advanced": true,
    "display_name": "Use Double Brackets",
    "dynamic": false,
    "info": "Use {{variable}} syntax instead of {variable}.",
    "list": false,
    "list_add_label": "Add More",
    "name": "use_double_brackets",
    "override_skip": false,
    "placeholder": "",
    "real_time_refresh": true,
    "required": false,
    "show": true,
    "title_case": false,
    "tool_mode": false,
    "trace_as_metadata": true,
    "track_in_telemetry": true,
    "type": "bool",
    "value": false
   }
  },
  "tool_mode": false
 },
 "showNode": true,
 "type": "Prompt Template"
```

```
  },
  "dragging": false,
  "id": "Prompt Template-TU3sw",
  "measured": {
   "height": 346,
   "width": 320
  },
  "position": {
   "x": -441.103475248301,
   "y": 103.89616922901799
  },
  "selected": false,
  "type": "genericNode"
 },
 {
  "data": {
   "id": "GroqModel-YiI97",
   "node": {
    "base_classes": [
     "LanguageModel",
     "Message"
    ],
    "beta": false,
    "conditional_paths": [],
    "custom_fields": {},
    "description": "Generate text using Groq.",
    "display_name": "Groq",
    "documentation": "",
```

```
"edited": false,
"field_order": [
  "input_value",
  "system_message",
  "stream",
  "api_key",
  "base_url",
  "max_tokens",
  "temperature",
  "n",
  "model_name",
  "tool_model_enabled"
],
"frozen": false,
"icon": "Groq",
"last_updated": "2026-04-30T09:59:02.336Z",
"legacy": false,
"lf_version": "1.8.2",
"metadata": {
  "code_hash": "2aee7de6ee88",
  "dependencies": {
    "dependencies": [
      {
        "name": "pydantic",
        "version": "2.11.10"
      },
      {
        "name": "lfx",
```

```
      "version": null
     },
     {
      "name": "langchain_groq",
      "version": "0.2.1"
     }
    ],
    "total_dependencies": 3
   },
   "keywords": [
    "model",
    "llm",
    "language model",
    "large language model"
   ],
   "module": "lfx.components.groq.groq.GroqModel"
  },
  "minimized": false,
  "output_types": [],
  "outputs": [
   {
    "allows_loop": false,
    "cache": true,
    "display_name": "Model Response",
    "group_outputs": false,
    "loop_types": null,
    "method": "text_response",
    "name": "text_output",
```

```
    "options": null,
    "required_inputs": null,
    "selected": "Message",
    "tool_mode": true,
    "types": [
     "Message"
    ],
    "value": "__UNDEFINED__"
   },
   {
    "allows_loop": false,
    "cache": true,
    "display_name": "Language Model",
    "group_outputs": false,
    "loop_types": null,
    "method": "build_model",
    "name": "model_output",
    "options": null,
    "required_inputs": null,
    "selected": "LanguageModel",
    "tool_mode": true,
    "types": [
     "LanguageModel"
    ],
    "value": "__UNDEFINED__"
   }
  ],
  "pinned": false,
```

```
"template": {
 "_frontend_node_flow_id": {
  "value": "efca400a-280a-4dfb-887a-fd5eac1eaa2b"
 },
 "_frontend_node_folder_id": {
  "value": "4926ca03-8f51-4362-84bf-66c50fb9f15c"
 },
 "_type": "Component",
 "api_key": {
  "_input_type": "SecretStrInput",
  "advanced": false,
  "display_name": "Groq API Key",
  "dynamic": false,
  "info": "API key for the Groq API.",
  "input_types": [],
  "load_from_db": true,
  "name": "api_key",
  "override_skip": false,
  "password": true,
  "placeholder": "",
  "real_time_refresh": true,
  "required": false,
  "show": true,
  "title_case": false,
  "track_in_telemetry": false,
  "type": "str",
  "value": "Groqkey"
 },
```

```
"base_url": {
 "_input_type": "MessageTextInput",
 "advanced": true,
 "display_name": "Groq API Base",
 "dynamic": false,
 "info": "Base URL path for API requests, leave blank if not using a proxy or service emulator.",
 "input_types": [
  "Message"
 ],
 "list": false,
 "list_add_label": "Add More",
 "load_from_db": false,
 "name": "base_url",
 "override_skip": false,
 "placeholder": "",
 "real_time_refresh": true,
 "required": false,
 "show": true,
 "title_case": false,
 "tool_mode": false,
 "trace_as_input": true,
 "trace_as_metadata": true,
 "track_in_telemetry": false,
 "type": "str",
 "value": "https://api.groq.com"
},
"code": {
```

"advanced": true,

"dynamic": true,

"fileTypes": [],

"file_path": "",

"info": "",

"list": false,

"load_from_db": false,

"multiline": true,

"name": "code",

"password": false,

"placeholder": "",

"required": true,

"show": true,

"title_case": false,

"type": "code",

"value": "from pydantic.v1 import SecretStr\n\nfrom lfx.base.models.groq_constants import GROQ_MODELS\nfrom lfx.base.models.groq_model_discovery import get_groq_models\nfrom lfx.base.models.model import LCModelComponent\nfrom lfx.field_typing import LanguageModel\nfrom lfx.field_typing.range_spec import RangeSpec\nfrom lfx.io import BoolInput, DropdownInput, IntInput, MessageTextInput, SecretStrInput, SliderInput\nfrom lfx.log.logger import logger\n\n\nclass GroqModel(LCModelComponent):\n display_name: str = \"Groq\"\n description: str = \"Generate text using Groq.\"\n icon = \"Groq\"\n name = \"GroqModel\"\n\n inputs = [\n *LCModelComponent.get_base_inputs(),\n SecretStrInput(\n name=\"api_key\", display_name=\"Groq API Key\", info=\"API key for the Groq API.\", real_time_refresh=True\n),\n MessageTextInput(\n name=\"base_url\",\n display_name=\"Groq API Base\",\n info=\"Base URL path for API requests, leave blank if not using a proxy or service emulator.\",\n advanced=True,\n value=\"https://api.groq.com\",\n real_time_refresh=True,\n),\n IntInput(\n name=\"max_tokens\",\n display_name=\"Max Output Tokens\",\n info=\"The maximum number of tokens to generate.\",\n advanced=True,\n),\n SliderInput(\n name=\"temperature\",\n display_name=\"Temperature\",\n value=0.1,\n info=\"Run inference with this temperature. Must by in the closed

```
interval [0.0, 1.0].\",\n            range_spec=RangeSpec(min=0, max=1, step=0.01),\n            advanced=True,\n        ),\n        IntInput(\n            name=\"n\",\n            display_name=\"N\",\n            info=\"Number of chat completions to generate for each prompt. \"\n            \"Note that the API may not return the full n completions if duplicates are generated.\",\n            advanced=True,\n        ),\n        DropdownInput(\n            name=\"model_name\",\n            display_name=\"Model\",\n            info=\"The name of the model to use. Add your Groq API key to access additional available models.\",\n            options=GROQ_MODELS,\n            value=GROQ_MODELS[0],\n            refresh_button=True,\n            combobox=True,\n        ),\n        BoolInput(\n            name=\"tool_model_enabled\",\n            display_name=\"Enable Tool Models\",\n            info=(\n                \"Select if you want to use models that can work with tools. If yes, only those models will be shown.\"\n            ),\n            advanced=False,\n            value=False,\n            real_time_refresh=True,\n        ),\n    ]\n\n    def get_models(self, *, tool_model_enabled: bool | None = None) -> list[str]:\n        \"\"\"Get available Groq models using the dynamic discovery system.\n\n        This method uses the groq_model_discovery module which:\n        - Fetches models directly from Groq API\n        - Automatically tests tool calling support\n        - Caches results for 24 hours\n        - Falls back to hardcoded list if API fails\n\n        Args:\n            tool_model_enabled: If True, only return models that support tool calling\n\n        Returns:\n            List of available model IDs\n        \"\"\"\n        try:\n            # Get models with metadata from dynamic discovery system\n            api_key = self.api_key if hasattr(self, \"api_key\") and self.api_key else None\n            models_metadata = get_groq_models(api_key=api_key)\n\n            # Filter out non-LLM models (audio, TTS, guards)\n            model_ids = [\n                model_id for model_id, metadata in models_metadata.items() if not metadata.get(\"not_supported\", False)\n            ]\n\n            # Filter by tool calling support if requested\n            if tool_model_enabled:\n                model_ids = [model_id for model_id in model_ids if models_metadata[model_id].get(\"tool_calling\", False)]\n                logger.info(f\"Loaded {len(model_ids)} Groq models with tool calling support\")\n            else:\n                logger.info(f\"Loaded {len(model_ids)} Groq models\")\n        except (ValueError, KeyError, TypeError, ImportError) as e:\n            logger.exception(f\"Error getting model names: {e}\")\n            # Fallback to hardcoded list from groq_constants.py\n            return GROQ_MODELS\n        else:\n            return model_ids\n\n    def update_build_config(self, build_config: dict, field_value: str, field_name: str | None = None):\n        if field_name in {\"base_url\", \"model_name\", \"tool_model_enabled\", \"api_key\"} and field_value:\n            try:\n                if len(self.api_key) != 0:\n                    try:\n                        ids = self.get_models(tool_model_enabled=self.tool_model_enabled)\n                    except (ValueError, KeyError, TypeError, ImportError) as e:\n                        logger.exception(f\"Error getting model names: {e}\")\n                        ids = GROQ_MODELS\n                    build_config.setdefault(\"model_name\", {})\n                    build_config[\"model_name\"][\"options\"] = ids\n                    build_config[\"model_name\"].setdefault(\"value\", ids[0])\n            except (ValueError,
```

KeyError, TypeError, AttributeError) as e:\n msg = f\"Error getting model names: {e}\"\n raise ValueError(msg) from e\n return build_config\n\n def build_model(self) -> LanguageModel: # type: ignore[type-var]\n try:\n from langchain_groq import ChatGroq\n except ImportError as e:\n msg = \"langchain-groq is not installed. Please install it with `pip install langchain-groq`.\"\n raise ImportError(msg) from e\n\n return ChatGroq(\n model=self.model_name,\n max_tokens=self.max_tokens or None,\n temperature=self.temperature,\n base_url=self.base_url,\n n=self.n or 1,\n api_key=SecretStr(self.api_key).get_secret_value(),\n streaming=self.stream,\n)\n"

},

"input_value": {

"_input_type": "MessageInput",

"advanced": false,

"display_name": "Input",

"dynamic": false,

"info": "",

"input_types": [

"Message"

],

"list": false,

"list_add_label": "Add More",

"load_from_db": false,

"name": "input_value",

"override_skip": false,

"placeholder": "",

"required": false,

"show": true,

"title_case": false,

"tool_mode": false,

"trace_as_input": true,

```
  "trace_as_metadata": true,
  "track_in_telemetry": false,
  "type": "str",
  "value": ""
 },
 "is_refresh": false,
 "max_tokens": {
  "_input_type": "IntInput",
  "advanced": true,
  "display_name": "Max Output Tokens",
  "dynamic": false,
  "info": "The maximum number of tokens to generate.",
  "list": false,
  "list_add_label": "Add More",
  "name": "max_tokens",
  "override_skip": false,
  "placeholder": "",
  "required": false,
  "show": true,
  "title_case": false,
  "tool_mode": false,
  "trace_as_metadata": true,
  "track_in_telemetry": true,
  "type": "int",
  "value": 0
 },
 "model_name": {
  "_input_type": "DropdownInput",
```

```
          "advanced": false,
          "combobox": true,
          "dialog_inputs": {},
          "display_name": "Model",
          "dynamic": false,
          "external_options": {},
          "info": "The name of the model to use. Add your Groq API key to access additional available models.",
          "name": "model_name",
          "options": [
            "llama-3.1-8b-instant",
            "llama-3.3-70b-versatile"
          ],
          "options_metadata": [],
          "override_skip": false,
          "placeholder": "",
          "refresh_button": true,
          "required": false,
          "show": true,
          "title_case": false,
          "toggle": false,
          "tool_mode": false,
          "trace_as_metadata": true,
          "track_in_telemetry": true,
          "type": "str",
          "value": "openai/gpt-oss-120b"
        },
        "n": {
```

```
"_input_type": "IntInput",
"advanced": true,
"display_name": "N",
"dynamic": false,
"info": "Number of chat completions to generate for each prompt. Note that the API may not return the full n completions if duplicates are generated.",
"list": false,
"list_add_label": "Add More",
"name": "n",
"override_skip": false,
"placeholder": "",
"required": false,
"show": true,
"title_case": false,
"tool_mode": false,
"trace_as_metadata": true,
"track_in_telemetry": true,
"type": "int",
"value": 0
},
"stream": {
"_input_type": "BoolInput",
"advanced": true,
"display_name": "Stream",
"dynamic": false,
"info": "Stream the response from the model. Streaming works only in Chat.",
"list": false,
"list_add_label": "Add More",
```

```
  "name": "stream",
  "override_skip": false,
  "placeholder": "",
  "required": false,
  "show": true,
  "title_case": false,
  "tool_mode": false,
  "trace_as_metadata": true,
  "track_in_telemetry": true,
  "type": "bool",
  "value": false
 },
 "system_message": {
  "_input_type": "MultilineInput",
  "advanced": false,
  "ai_enabled": false,
  "copy_field": false,
  "display_name": "System Message",
  "dynamic": false,
  "info": "System message to pass to the model.",
  "input_types": [
   "Message"
  ],
  "list": false,
  "list_add_label": "Add More",
  "load_from_db": false,
  "multiline": true,
  "name": "system_message",
```

```
        "override_skip": false,
        "password": false,
        "placeholder": "",
        "required": false,
        "show": true,
        "title_case": false,
        "tool_mode": false,
        "trace_as_input": true,
        "trace_as_metadata": true,
        "track_in_telemetry": false,
        "type": "str",
        "value": ""
      },
      "temperature": {
        "_input_type": "SliderInput",
        "advanced": true,
        "display_name": "Temperature",
        "dynamic": false,
        "info": "Run inference with this temperature. Must by in the closed interval [0.0, 1.0].",
        "max_label": "",
        "max_label_icon": "",
        "min_label": "",
        "min_label_icon": "",
        "name": "temperature",
        "override_skip": false,
        "placeholder": "",
        "range_spec": {
```

```
      "max": 1,
      "min": 0,
      "step": 0.01,
      "step_type": "float"
    },
    "required": false,
    "show": true,
    "slider_buttons": false,
    "slider_buttons_options": [],
    "slider_input": false,
    "title_case": false,
    "tool_mode": false,
    "track_in_telemetry": false,
    "type": "slider",
    "value": 0.1
  },
  "tool_model_enabled": {
    "_input_type": "BoolInput",
    "advanced": false,
    "display_name": "Enable Tool Models",
    "dynamic": false,
    "info": "Select if you want to use models that can work with tools. If yes, only those models will be shown.",
    "list": false,
    "list_add_label": "Add More",
    "name": "tool_model_enabled",
    "override_skip": false,
    "placeholder": "",
```

```
        "real_time_refresh": true,
        "required": false,
        "show": true,
        "title_case": false,
        "tool_mode": false,
        "trace_as_metadata": true,
        "track_in_telemetry": true,
        "type": "bool",
        "value": false
      }
    },
    "tool_mode": false
  },
  "selected_output": "text_output",
  "showNode": true,
  "type": "GroqModel"
},
"dragging": false,
"id": "GroqModel-YiI97",
"measured": {
  "height": 493,
  "width": 320
},
"position": {
  "x": -44.98677810515264,
  "y": 71.29719837266862
},
"selected": false,
```

```
  "type": "genericNode"
},
{
  "data": {
    "id": "File-uI9B7",
    "node": {
      "base_classes": [
        "Message"
      ],
      "beta": false,
      "category": "files_and_knowledge",
      "conditional_paths": [],
      "custom_fields": {},
      "description": "Loads and returns the content from uploaded files.",
      "display_name": "Read File",
      "documentation": "https://docs.langflow.org/read-file",
      "edited": false,
      "field_order": [
        "storage_location",
        "path",
        "file_path",
        "separator",
        "silent_errors",
        "delete_server_file_after_processing",
        "ignore_unsupported_extensions",
        "ignore_unspecified_files",
        "file_path_str",
        "aws_access_key_id",
```

```
  "aws_secret_access_key",
  "bucket_name",
  "aws_region",
  "s3_file_key",
  "service_account_key",
  "file_id",
  "advanced_mode",
  "pipeline",
  "ocr_engine",
  "md_image_placeholder",
  "md_page_break_placeholder",
  "doc_key",
  "use_multithreading",
  "concurrency_multithreading",
  "markdown"
 ],
 "frozen": false,
 "icon": "file-text",
 "key": "File",
 "last_updated": "2026-04-30T10:18:05.120Z",
 "legacy": false,
 "lf_version": "1.8.2",
 "metadata": {
  "code_hash": "12a5841f1a03",
  "dependencies": {
   "dependencies": [
    {
     "name": "lfx",
```

```
      "version": null
     },
     {
      "name": "langchain_core",
      "version": "0.3.83"
     },
     {
      "name": "pydantic",
      "version": "2.11.10"
     },
     {
      "name": "googleapiclient",
      "version": "2.154.0"
     }
    ],
    "total_dependencies": 4
   },
   "module": "lfx.components.files_and_knowledge.file.FileComponent"
},
"minimized": false,
"output_types": [],
"outputs": [
 {
   "allows_loop": false,
   "cache": true,
   "display_name": "Raw Content",
   "group_outputs": false,
   "loop_types": null,
```

```
  "method": "load_files_message",
  "name": "message",
  "options": null,
  "required_inputs": null,
  "selected": "Message",
  "tool_mode": true,
  "types": [
   "Message"
  ],
  "value": "__UNDEFINED__"
 },
 {
  "allows_loop": false,
  "cache": true,
  "display_name": "File Path",
  "group_outputs": false,
  "hidden": null,
  "loop_types": null,
  "method": "load_files_path",
  "name": "path",
  "options": null,
  "required_inputs": null,
  "selected": "Message",
  "tool_mode": true,
  "types": [
   "Message"
  ],
  "value": "__UNDEFINED__"
```

```
      }
    ],
    "pinned": false,
    "score": 0.007568328950209746,
    "template": {
     "_frontend_node_flow_id": {
      "value": "efca400a-280a-4dfb-887a-fd5eac1eaa2b"
     },
     "_frontend_node_folder_id": {
      "value": "4926ca03-8f51-4362-84bf-66c50fb9f15c"
     },
     "_type": "Component",
     "advanced_mode": {
      "_input_type": "BoolInput",
      "advanced": false,
      "display_name": "Advanced Parser",
      "dynamic": false,
      "info": "Enable advanced document processing and export with Docling for PDFs, images, and office documents. Note that advanced document processing can consume significant resources.",
      "list": false,
      "list_add_label": "Add More",
      "name": "advanced_mode",
      "override_skip": false,
      "placeholder": "",
      "real_time_refresh": true,
      "required": false,
      "show": true,
      "title_case": false,
```

```
  "tool_mode": false,
  "trace_as_metadata": true,
  "track_in_telemetry": true,
  "type": "bool",
  "value": false
},
"aws_access_key_id": {
  "_input_type": "SecretStrInput",
  "advanced": false,
  "display_name": "AWS Access Key ID",
  "dynamic": false,
  "info": "AWS Access key ID.",
  "input_types": [],
  "load_from_db": false,
  "name": "aws_access_key_id",
  "override_skip": false,
  "password": true,
  "placeholder": "",
  "required": true,
  "show": false,
  "title_case": false,
  "track_in_telemetry": false,
  "type": "str",
  "value": ""
},
"aws_region": {
  "_input_type": "StrInput",
  "advanced": false,
```

```
  "display_name": "AWS Region",
  "dynamic": false,
  "info": "AWS region (e.g., us-east-1, eu-west-1).",
  "list": false,
  "list_add_label": "Add More",
  "load_from_db": false,
  "name": "aws_region",
  "override_skip": false,
  "placeholder": "",
  "required": false,
  "show": false,
  "title_case": false,
  "tool_mode": false,
  "trace_as_metadata": true,
  "track_in_telemetry": false,
  "type": "str",
  "value": ""
 },
 "aws_secret_access_key": {
  "_input_type": "SecretStrInput",
  "advanced": false,
  "display_name": "AWS Secret Key",
  "dynamic": false,
  "info": "AWS Secret Key.",
  "input_types": [],
  "load_from_db": false,
  "name": "aws_secret_access_key",
  "override_skip": false,
```

```
  "password": true,
  "placeholder": "",
  "required": true,
  "show": false,
  "title_case": false,
  "track_in_telemetry": false,
  "type": "str",
  "value": ""
},
"bucket_name": {
  "_input_type": "StrInput",
  "advanced": false,
  "display_name": "S3 Bucket Name",
  "dynamic": false,
  "info": "Enter the name of the S3 bucket.",
  "list": false,
  "list_add_label": "Add More",
  "load_from_db": false,
  "name": "bucket_name",
  "override_skip": false,
  "placeholder": "",
  "required": true,
  "show": false,
  "title_case": false,
  "tool_mode": false,
  "trace_as_metadata": true,
  "track_in_telemetry": false,
  "type": "str",
```

```
          "value": ""
        },
        "code": {
          "advanced": true,
          "dynamic": true,
          "fileTypes": [],
          "file_path": "",
          "info": "",
          "list": false,
          "load_from_db": false,
          "multiline": true,
          "name": "code",
          "password": false,
          "placeholder": "",
          "required": true,
          "show": true,
          "title_case": false,
          "type": "code",
          "value": "\"\"\"Enhanced file component with Docling support and process isolation.\n\nNotes:\n-----\n- ALL Docling parsing/export runs in a separate OS process to prevent memory\n  growth and native library state from impacting the main Langflow process.\n- Standard text/structured parsing continues to use existing BaseFileComponent\n  utilities (and optional threading via `parallel_load_data`).\n\"\"\"\n\nfrom __future__ import annotations\n\nimport contextlib\nimport json\nimport subprocess\nimport sys\nimport textwrap\nfrom copy import deepcopy\nfrom pathlib import Path\nfrom tempfile import NamedTemporaryFile\nfrom typing import Any\n\nfrom lfx.base.data.base_file import BaseFileComponent\nfrom lfx.base.data.storage_utils import parse_storage_path, read_file_bytes, validate_image_content_type\nfrom lfx.base.data.utils import TEXT_FILE_TYPES, parallel_load_data, parse_text_file_to_data\nfrom lfx.inputs import SortableListInput\nfrom lfx.inputs.inputs import DropdownInput, MessageTextInput, StrInput\nfrom lfx.io import BoolInput, FileInput, IntInput, Output, SecretStrInput\nfrom
```

lfx.schema.data import Data\nfrom lfx.schema.dataframe import DataFrame # noqa: TC001\nfrom lfx.schema.message import Message\nfrom lfx.services.deps import get_settings_service, get_storage_service\nfrom lfx.utils.async_helpers import run_until_complete\nfrom lfx.utils.validate_cloud import is_astra_cloud_environment\n\n\ndef _get_storage_location_options():\n   \"\"\"Get storage location options, filtering out Local if in Astra cloud environment.\"\"\"\n   all_options = [{\"name\": \"AWS\", \"icon\": \"Amazon\"}, {\"name\": \"Google Drive\", \"icon\": \"google\"}]\n   if is_astra_cloud_environment():\n   return all_options\n   return [{\"name\": \"Local\", \"icon\": \"hard-drive\"}, *all_options]\n\n\nclass FileComponent(BaseFileComponent):\n   \"\"\"File component with optional Docling processing (isolated in a subprocess).\"\"\"\n\n   display_name = \"Read File\"\n   # description is now a dynamic property - see get_tool_description()\n   _base_description = \"Loads content from one or more files.\"\n   documentation: str = \"https://docs.langflow.org/read-file\"\n   icon = \"file-text\"\n   name = \"File\"\n   add_tool_output = True # Enable tool mode toggle without requiring tool_mode inputs\n\n   # Extensions that can be processed without Docling (using standard text parsing)\n   TEXT_EXTENSIONS = TEXT_FILE_TYPES\n\n   # Extensions that require Docling for processing (images, advanced office formats, etc.)\n   DOCLING_ONLY_EXTENSIONS = [\n   \"adoc\",\n   \"asciidoc\",\n   \"asc\",\n   \"bmp\",\n   \"dotx\",\n   \"dotm\",\n   \"docm\",\n   \"jpg\",\n   \"jpeg\",\n   \"png\",\n   \"potx\",\n   \"ppsx\",\n   \"pptm\",\n   \"potm\",\n   \"ppsm\",\n   \"pptx\",\n   \"tiff\",\n   \"xls\",\n   \"xlsx\",\n   \"xhtml\",\n   \"webp\",\n   ]\n\n   # Docling-supported/compatible extensions; TEXT_FILE_TYPES are supported by the base loader.\n   VALID_EXTENSIONS = [\n   *TEXT_EXTENSIONS,\n   *DOCLING_ONLY_EXTENSIONS,\n   ]\n\n   # Fixed export settings used when markdown export is requested.\n   EXPORT_FORMAT = \"Markdown\"\n   IMAGE_MODE = \"placeholder\"\n\n   _base_inputs = deepcopy(BaseFileComponent.get_base_inputs())\n\n   for input_item in _base_inputs:\n   if isinstance(input_item, FileInput) and input_item.name == \"path\":\n   input_item.real_time_refresh = True\n   input_item.tool_mode = False # Disable tool mode for file upload input\n   input_item.required = False # Make it optional so it doesn't error in tool mode\n   break\n\n   inputs = [\n   SortableListInput(\n   name=\"storage_location\",\n   display_name=\"Storage Location\",\n   placeholder=\"Select Location\",\n   info=\"Choose where to read the file from.\",\n   options=_get_storage_location_options(),\n   real_time_refresh=True,\n   limit=1,\n   value=[{\"name\": \"Local\", \"icon\": \"hard-drive\"}],\n   advanced=True,\n   ),\n   *_base_inputs,\n   StrInput(\n   name=\"file_path_str\",\n   display_name=\"File Path\",\n   info=(\n   \"Path to the file to read. Used when component is called as a tool. \"\n   \"If not provided, will use the uploaded file from 'path' input.\"\n   ),\n   show=False,\n   advanced=True,\n   tool_mode=True, # Required for Toolset toggle, but _get_tools()

ignores this parameter\n required=False,\n),\n # AWS S3 specific inputs\n SecretStrInput(\n name=\"aws_access_key_id\",\n display_name=\"AWS Access Key ID\",\n info=\"AWS Access key ID.\",\n show=False,\n advanced=False,\n required=True,\n),\n SecretStrInput(\n name=\"aws_secret_access_key\",\n display_name=\"AWS Secret Key\",\n info=\"AWS Secret Key.\",\n show=False,\n advanced=False,\n required=True,\n),\n StrInput(\n name=\"bucket_name\",\n display_name=\"S3 Bucket Name\",\n info=\"Enter the name of the S3 bucket.\",\n show=False,\n advanced=False,\n required=True,\n),\n StrInput(\n name=\"aws_region\",\n display_name=\"AWS Region\",\n info=\"AWS region (e.g., us-east-1, eu-west-1).\",\n show=False,\n advanced=False,\n),\n StrInput(\n name=\"s3_file_key\",\n display_name=\"S3 File Key\",\n info=\"The key (path) of the file in S3 bucket.\",\n show=False,\n advanced=False,\n required=True,\n),\n # Google Drive specific inputs\n SecretStrInput(\n name=\"service_account_key\",\n display_name=\"GCP Credentials Secret Key\",\n info=\"Your Google Cloud Platform service account JSON key as a secret string (complete JSON content).\",\n show=False,\n advanced=False,\n required=True,\n),\n StrInput(\n name=\"file_id\",\n display_name=\"Google Drive File ID\",\n info=(\"The Google Drive file ID to read. The file must be shared with the service account email.\"),\n show=False,\n advanced=False,\n required=True,\n),\n BoolInput(\n name=\"advanced_mode\",\n display_name=\"Advanced Parser\",\n value=False,\n real_time_refresh=True,\n info=(\n \"Enable advanced document processing and export with Docling for PDFs, images, and office documents. \"\n \"Note that advanced document processing can consume significant resources.\"\n),\n # Disabled in cloud\n show=not is_astra_cloud_environment(),\n),\n DropdownInput(\n name=\"pipeline\",\n display_name=\"Pipeline\",\n info=\"Docling pipeline to use\",\n options=[\"standard\", \"vlm\"],\n value=\"standard\",\n advanced=True,\n real_time_refresh=True,\n),\n DropdownInput(\n name=\"ocr_engine\",\n display_name=\"OCR Engine\",\n info=\"OCR engine to use. Only available when pipeline is set to 'standard'.\",\n options=[\"None\", \"easyocr\"],\n value=\"easyocr\",\n show=False,\n advanced=True,\n),\n StrInput(\n name=\"md_image_placeholder\",\n display_name=\"Image placeholder\",\n info=\"Specify the image placeholder for markdown exports.\",\n value=\"<!-- image -->\",\n advanced=True,\n show=False,\n),\n StrInput(\n name=\"md_page_break_placeholder\",\n display_name=\"Page break placeholder\",\n info=\"Add this placeholder between pages in the markdown output.\",\n value=\"\",\n advanced=True,\n show=False,\n),\n MessageTextInput(\n name=\"doc_key\",\n display_name=\"Doc Key\",\n info=\"The key to use for the DoclingDocument

```
column.\",\n value=\"doc\",\n advanced=True,\n show=False,\n ),\n # Deprecated input retained for backward-compatibility.\n BoolInput(\n name=\"use_multithreading\",\n display_name=\"[Deprecated] Use Multithreading\",\n advanced=True,\n value=True,\n info=\"Set 'Processing Concurrency' greater than 1 to enable multithreading.\",\n ),\n IntInput(\n name=\"concurrency_multithreading\",\n display_name=\"Processing Concurrency\",\n advanced=True,\n info=\"When multiple files are being processed, the number of files to process concurrently.\",\n value=1,\n ),\n BoolInput(\n name=\"markdown\",\n display_name=\"Markdown Export\",\n info=\"Export processed documents to Markdown format. Only available when advanced mode is enabled.\",\n value=False,\n show=False,\n ),\n ]\n\n outputs = [\n Output(display_name=\"Raw Content\", name=\"message\", method=\"load_files_message\", tool_mode=True),\n ]\n\n # ------------------------------ Tool description with file names --------------\n\n def get_tool_description(self) -> str:\n \"\"\"Return a dynamic description that includes the names of uploaded files.\n\n This helps the Agent understand which files are available to read.\n \"\"\"\n base_description = \"Loads and returns the content from uploaded files.\"\n\n # Get the list of uploaded file paths\n file_paths = getattr(self, \"path\", None)\n if not file_paths:\n return base_description\n\n # Ensure it's a list\n if not isinstance(file_paths, list):\n file_paths = [file_paths]\n\n # Extract just the file names from the paths\n file_names = []\n for fp in file_paths:\n if fp:\n name = Path(fp).name\n file_names.append(name)\n\n if file_names:\n files_str = \", \".join(file_names)\n return f\"{base_description} Available files: {files_str}. Call this tool to read these files.\"\n\n return base_description\n\n @property\n def description(self) -> str:\n \"\"\"Dynamic description property that includes uploaded file names.\"\"\"\n return self.get_tool_description()\n\n async def _get_tools(self) -> list:\n \"\"\"Override to create a tool without parameters.\n\n The Read File component should use the files already uploaded via UI,\n not accept file paths from the Agent (which wouldn't know the internal paths).\n \"\"\"\n from langchain_core.tools import StructuredTool\n from pydantic import BaseModel\n\n # Empty schema - no parameters needed\n class EmptySchema(BaseModel):\n \"\"\"No parameters required - uses pre-uploaded files.\"\"\"\n\n async def read_files_tool() -> str:\n \"\"\"Read the content of uploaded files.\"\"\"\n try:\n result = self.load_files_message()\n if hasattr(result, \"get_text\"):\n return result.get_text()\n if hasattr(result, \"text\"):\n return result.text\n return str(result)\n except (FileNotFoundError, ValueError, OSError, RuntimeError) as e:\n return f\"Error reading files: {e}\"\n\n description = self.get_tool_description()\n\n tool = StructuredTool(\n name=\"load_files_message\",\n description=description,\n coroutine=read_files_tool,\n args_schema=EmptySchema,\n
```

```
handle_tool_error=True,\n        tags=[\"load_files_message\"],\n        metadata={\n
\"display_name\": \"Read File\",\n            \"display_description\": description,\n
},\n    )\n\n    return [tool]\n\n  # ----------------------------- UI helpers ---------------------
-----------------\n\n  def _path_value(self, template: dict) -> list[str]:\n    \"\"\"Return the
list of currently selected file paths from the template.\"\"\"\n    return
template.get(\"path\", {}).get(\"file_path\", [])\n\n  def
_disable_docling_fields_in_cloud(self, build_config: dict[str, Any]) -> None:\n
\"\"\"Disable all Docling-related fields in cloud environments.\"\"\"\n    if
\"advanced_mode\" in build_config:\n        build_config[\"advanced_mode\"][\"show\"]
= False\n        build_config[\"advanced_mode\"][\"value\"] = False\n    # Hide all
Docling-related fields\n    docling_fields = (\"pipeline\", \"ocr_engine\", \"doc_key\",
\"md_image_placeholder\", \"md_page_break_placeholder\")\n    for field in
docling_fields:\n        if field in build_config:\n            build_config[field][\"show\"] =
False\n    # Also disable OCR engine specifically\n    if \"ocr_engine\" in
build_config:\n        build_config[\"ocr_engine\"][\"value\"] = \"None\"\n\n  def
update_build_config(\n    self,\n    build_config: dict[str, Any],\n    field_value:
Any,\n    field_name: str | None = None,\n  ) -> dict[str, Any]:\n    \"\"\"Show/hide
Advanced Parser and related fields based on selection context.\"\"\"\n    # Update
storage location options dynamically based on cloud environment\n    if
\"storage_location\" in build_config:\n        updated_options =
_get_storage_location_options()\n        build_config[\"storage_location\"][\"options\"] =
updated_options\n\n    # Handle storage location selection\n    if field_name ==
\"storage_location\":\n        # Extract selected storage location\n        selected =
[location[\"name\"] for location in field_value] if isinstance(field_value, list) else []\n\n
# Hide all storage-specific fields first\n        storage_fields = [\n
\"aws_access_key_id\",\n            \"aws_secret_access_key\",\n
\"bucket_name\",\n            \"aws_region\",\n            \"s3_file_key\",\n
\"service_account_key\",\n            \"file_id\",\n        ]\n\n        for f_name in
storage_fields:\n            if f_name in build_config:\n
build_config[f_name][\"show\"] = False\n\n        # Show fields based on selected
storage location\n        if len(selected) == 1:\n            location = selected[0]\n\n
if location == \"Local\":\n                # Show file upload input for local storage\n                if
\"path\" in build_config:\n                    build_config[\"path\"][\"show\"] = True\n\n
elif location == \"AWS\":\n                # Hide file upload input, show AWS fields\n
if \"path\" in build_config:\n                    build_config[\"path\"][\"show\"] = False\n\n
aws_fields = [\n                    \"aws_access_key_id\",\n
\"aws_secret_access_key\",\n                    \"bucket_name\",\n
\"aws_region\",\n                    \"s3_file_key\",\n                ]\n                for f_name in
aws_fields:\n                    if f_name in build_config:\n
build_config[f_name][\"show\"] = True\n
build_config[f_name][\"advanced\"] = False\n\n            elif location == \"Google
```

```
Drive\":\n                # Hide file upload input, show Google Drive fields\n                if \"path\" in build_config:\n                    build_config[\"path\"][\"show\"] = False\n\n                gdrive_fields = [\"service_account_key\", \"file_id\"]\n                for f_name in gdrive_fields:\n                    if f_name in build_config:\n                        build_config[f_name][\"show\"] = True\n                        build_config[f_name][\"advanced\"] = False\n            # No storage location selected - show file upload by default\n            elif \"path\" in build_config:\n                build_config[\"path\"][\"show\"] = True\n\n            return build_config\n\n        if field_name == \"path\":\n            paths = self._path_value(build_config)\n\n            # Disable in cloud environments\n            if is_astra_cloud_environment():\n                self._disable_docling_fields_in_cloud(build_config)\n            else:\n                # If all files can be processed by docling, do so\n                allow_advanced = all(not file_path.endswith((\".csv\", \".xlsx\", \".parquet\")) for file_path in paths)\n                build_config[\"advanced_mode\"][\"show\"] = allow_advanced\n                if not allow_advanced:\n                    build_config[\"advanced_mode\"][\"value\"] = False\n                docling_fields = (\n                    \"pipeline\",\n                    \"ocr_engine\",\n                    \"doc_key\",\n                    \"md_image_placeholder\",\n                    \"md_page_break_placeholder\",\n                )\n                for field in docling_fields:\n                    if field in build_config:\n                        build_config[field][\"show\"] = False\n\n        # Docling Processing\n        elif field_name == \"advanced_mode\":\n            # Disable in cloud environments - don't show Docling fields even if advanced_mode is toggled\n            if is_astra_cloud_environment():\n                self._disable_docling_fields_in_cloud(build_config)\n            else:\n                docling_fields = (\n                    \"pipeline\",\n                    \"ocr_engine\",\n                    \"doc_key\",\n                    \"md_image_placeholder\",\n                    \"md_page_break_placeholder\",\n                )\n                for field in docling_fields:\n                    if field in build_config:\n                        build_config[field][\"show\"] = bool(field_value)\n                        if field == \"pipeline\":\n                            build_config[field][\"advanced\"] = not bool(field_value)\n\n        elif field_name == \"pipeline\":\n            # Disable in cloud environments - don't show OCR engine even if pipeline is changed\n            if is_astra_cloud_environment():\n                self._disable_docling_fields_in_cloud(build_config)\n            elif field_value == \"standard\":\n                build_config[\"ocr_engine\"][\"show\"] = True\n                build_config[\"ocr_engine\"][\"value\"] = \"easyocr\"\n            else:\n                build_config[\"ocr_engine\"][\"show\"] = False\n                build_config[\"ocr_engine\"][\"value\"] = \"None\"\n\n        return build_config\n\n    def update_outputs(self, frontend_node: dict[str, Any], field_name: str, field_value: Any) -> dict[str, Any]:  # noqa: ARG002\n        \"\"\"Dynamically show outputs based on file count/type and advanced mode.\"\"\"\n        if field_name not in [\"path\", \"advanced_mode\", \"pipeline\"]:\n            return frontend_node\n\n        template = frontend_node.get(\"template\", {})\n        paths = self._path_value(template)\n        if not paths:\n            return frontend_node\n\n        frontend_node[\"outputs\"] = []\n        if
```

```
len(paths) == 1:\n            file_path = paths[0] if field_name == \"path\" else
frontend_node[\"template\"][\"path\"][\"file_path\"][0]\n            if
file_path.endswith((\".csv\", \".xlsx\", \".parquet\")):\n
frontend_node[\"outputs\"].append(\n                Output(\n
display_name=\"Structured Content\",\n                    name=\"dataframe\",\n
method=\"load_files_structured\",\n                tool_mode=True,\n                ),\n
)\n            elif file_path.endswith(\".json\"):\n
frontend_node[\"outputs\"].append(\n                Output(display_name=\"Structured
Content\", name=\"json\", method=\"load_files_json\", tool_mode=True),\n            )\n\n
advanced_mode = frontend_node.get(\"template\", {}).get(\"advanced_mode\",
{}).get(\"value\", False)\n            if advanced_mode:\n
frontend_node[\"outputs\"].append(\n                Output(\n
display_name=\"Structured Output\",\n                    name=\"advanced_dataframe\",\n
method=\"load_files_dataframe\",\n                tool_mode=True,\n                ),\n
)\n                frontend_node[\"outputs\"].append(\n                Output(\n
display_name=\"Markdown\", name=\"advanced_markdown\",
method=\"load_files_markdown\", tool_mode=True\n                ),\n            )\n
frontend_node[\"outputs\"].append(\n                Output(display_name=\"File Path\",
name=\"path\", method=\"load_files_path\", tool_mode=True),\n            )\n            else:\n
frontend_node[\"outputs\"].append(\n                Output(display_name=\"Raw Content\",
name=\"message\", method=\"load_files_message\", tool_mode=True),\n            )\n
frontend_node[\"outputs\"].append(\n                Output(display_name=\"File Path\",
name=\"path\", method=\"load_files_path\", tool_mode=True),\n            )\n        else:\n
# Multiple files => DataFrame output; advanced parser disabled\n
frontend_node[\"outputs\"].append(\n            Output(display_name=\"Files\",
name=\"dataframe\", method=\"load_files\", tool_mode=True)\n            )\n\n        return
frontend_node\n\n    # ---------------------------- Core processing ----------------------------
----\n\n    def _get_selected_storage_location(self) -> str:\n        \"\"\"Get the selected
storage location from the SortableListInput.\"\"\"\n        if hasattr(self,
\"storage_location\") and self.storage_location:\n            if
isinstance(self.storage_location, list) and len(self.storage_location) > 0:\n                return
self.storage_location[0].get(\"name\", \"\")\n            if isinstance(self.storage_location,
dict):\n                return self.storage_location.get(\"name\", \"\")\n        return \"Local\"  #
Default to Local if not specified\n\n    def _validate_and_resolve_paths(self) ->
list[BaseFileComponent.BaseFile]:\n        \"\"\"Override to handle file_path_str input
from tool mode and cloud storage.\n\n        Priority:\n        1. Cloud storage (AWS/Google
Drive) if selected\n        2. file_path_str (if provided by the tool call)\n        3. path
(uploaded file from UI)\n        \"\"\"\n        storage_location =
self._get_selected_storage_location()\n\n        # Handle AWS S3\n        if storage_location
== \"AWS\":\n            return self._read_from_aws_s3()\n\n        # Handle Google Drive\n
if storage_location == \"Google Drive\":\n            return
```

self._read_from_google_drive()\n\n # Handle Local storage\n # Check if file_path_str is provided (from tool mode)\n file_path_str = getattr(self, \"file_path_str\", None)\n if file_path_str:\n # Use the string path from tool mode\n from pathlib import Path\n\n from lfx.schema.data import Data\n\n # Use same resolution logic as BaseFileComponent (support storage paths)\n path_str = str(file_path_str)\n if parse_storage_path(path_str):\n try:\n resolved_path = Path(self.get_full_path(path_str))\n except (ValueError, AttributeError):\n resolved_path = Path(self.resolve_path(path_str))\n else:\n resolved_path = Path(self.resolve_path(path_str))\n\n if not resolved_path.exists():\n msg = f\"File or directory not found: {file_path_str}\"\n self.log(msg)\n if not self.silent_errors:\n raise ValueError(msg)\n return []\n\n data_obj = Data(data={self.SERVER_FILE_PATH_FIELDNAME: str(resolved_path)})\n return [BaseFileComponent.BaseFile(data_obj, resolved_path, delete_after_processing=False)]\n\n # Otherwise use the default implementation (uses path FileInput)\n return super()._validate_and_resolve_paths()\n\n def _read_from_aws_s3(self) -> list[BaseFileComponent.BaseFile]:\n \"\"\"Read file from AWS S3.\"\"\"\n from lfx.base.data.cloud_storage_utils import create_s3_client, validate_aws_credentials\n\n # Validate AWS credentials\n validate_aws_credentials(self)\n if not getattr(self, \"s3_file_key\", None):\n msg = \"S3 File Key is required\"\n raise ValueError(msg)\n\n # Create S3 client\n s3_client = create_s3_client(self)\n\n # Download file to temp location\n import tempfile\n\n # Get file extension from S3 key\n file_extension = Path(self.s3_file_key).suffix or \"\"\n\n with tempfile.NamedTemporaryFile(mode=\"wb\", suffix=file_extension, delete=False) as temp_file:\n temp_file_path = temp_file.name\n try:\n s3_client.download_fileobj(self.bucket_name, self.s3_file_key, temp_file)\n except Exception as e:\n # Clean up temp file on failure\n with contextlib.suppress(OSError):\n Path(temp_file_path).unlink()\n msg = f\"Failed to download file from S3: {e}\"\n raise RuntimeError(msg) from e\n\n # Create BaseFile object\n from lfx.schema.data import Data\n\n temp_path = Path(temp_file_path)\n data_obj = Data(data={self.SERVER_FILE_PATH_FIELDNAME: str(temp_path)})\n return [BaseFileComponent.BaseFile(data_obj, temp_path, delete_after_processing=True)]\n\n def _read_from_google_drive(self) -> list[BaseFileComponent.BaseFile]:\n \"\"\"Read file from Google Drive.\"\"\"\n import tempfile\n\n from googleapiclient.http import MediaIoBaseDownload\n\n from lfx.base.data.cloud_storage_utils import create_google_drive_service\n\n # Validate Google Drive credentials\n if not getattr(self, \"service_account_key\", None):\n msg = \"GCP Credentials Secret Key is required for Google Drive storage\"\n raise ValueError(msg)\n if not getattr(self, \"file_id\", None):\n msg = \"Google Drive File ID is required\"\n raise ValueError(msg)\n\n # Create

Google Drive service with read-only scope\n drive_service = create_google_drive_service(\n self.service_account_key, scopes=[\"https://www.googleapis.com/auth/drive.readonly\"]\n)\n\n # Get file metadata to determine file name and extension\n try:\n file_metadata = drive_service.files().get(fileId=self.file_id, fields=\"name,mimeType\").execute()\n file_name = file_metadata.get(\"name\", \"download\")\n except Exception as e:\n msg = (\n f\"Unable to access file with ID '{self.file_id}'. \"\n f\"Error: {e!s}. \"\n \"Please ensure: 1) The file ID is correct, 2) The file exists, \"\n \"3) The service account has been granted access to this file.\"\n)\n raise ValueError(msg) from e\n\n # Download file to temp location\n file_extension = Path(file_name).suffix or \"\"\n with tempfile.NamedTemporaryFile(mode=\"wb\", suffix=file_extension, delete=False) as temp_file:\n temp_file_path = temp_file.name\n try:\n request = drive_service.files().get_media(fileId=self.file_id)\n downloader = MediaIoBaseDownload(temp_file, request)\n done = False\n while not done:\n _status, done = downloader.next_chunk()\n except Exception as e:\n # Clean up temp file on failure\n with contextlib.suppress(OSError):\n Path(temp_file_path).unlink()\n msg = f\"Failed to download file from Google Drive: {e}\"\n raise RuntimeError(msg) from e\n\n # Create BaseFile object\n from lfx.schema.data import Data\n\n temp_path = Path(temp_file_path)\n data_obj = Data(data={self.SERVER_FILE_PATH_FIELDNAME: str(temp_path)})\n return [BaseFileComponent.BaseFile(data_obj, temp_path, delete_after_processing=True)]\n\n def _is_docling_compatible(self, file_path: str) -> bool:\n \"\"\"Lightweight extension gate for Docling-compatible types.\"\"\"\n docling_exts = (\n \".adoc\",\n \".asciidoc\",\n \".asc\",\n \".bmp\",\n \".csv\",\n \".dotx\",\n \".dotm\",\n \".docm\",\n \".docx\",\n \".htm\",\n \".html\",\n \".jpg\",\n \".jpeg\",\n \".json\",\n \".md\",\n \".pdf\",\n \".png\",\n \".potx\",\n \".ppsx\",\n \".pptm\",\n \".potm\",\n \".ppsm\",\n \".pptx\",\n \".tiff\",\n \".txt\",\n \".xls\",\n \".xlsx\",\n \".xhtml\",\n \".xml\",\n \".webp\",\n)\n return file_path.lower().endswith(docling_exts)\n\n async def _get_local_file_for_docling(self, file_path: str) -> tuple[str, bool]:\n \"\"\"Get a local file path for Docling processing, downloading from S3 if needed.\n\n Args:\n file_path: Either a local path or S3 key (format \"flow_id/filename\")\n\n Returns:\n tuple[str, bool]: (local_path, should_delete) where should_delete indicates\n if this is a temporary file that should be cleaned up\n \"\"\"\n settings = get_settings_service().settings\n if settings.storage_type == \"local\":\n return file_path, False\n\n # S3 storage - download to temp file\n parsed = parse_storage_path(file_path)\n if not parsed:\n msg = f\"Invalid S3 path format: {file_path}. Expected 'flow_id/filename'\"\n raise ValueError(msg)\n\n storage_service = get_storage_service()\n flow_id, filename = parsed\n\n # Get

```
file content from S3\n        content = await storage_service.get_file(flow_id, filename)\n\n
suffix = Path(filename).suffix\n        with NamedTemporaryFile(mode=\"wb\",
suffix=suffix, delete=False) as tmp_file:\n            tmp_file.write(content)\n
temp_path = tmp_file.name\n\n        return temp_path, True\n\n    def
_process_docling_in_subprocess(self, file_path: str) -> Data | None:\n        \"\"\"Run
Docling in a separate OS process and map the result to a Data object.\n\n        We avoid
multiprocessing pickling by launching `python -c \"<script>\"` and\n        passing JSON
config via stdin. The child prints a JSON result to stdout.\n\n        For S3 storage, the file is
downloaded to a temp file first.\n        \"\"\"\n        if not file_path:\n            return None\n\n
settings = get_settings_service().settings\n        if settings.storage_type == \"s3\":\n
local_path, should_delete =
run_until_complete(self._get_local_file_for_docling(file_path))\n        else:\n
local_path = file_path\n            should_delete = False\n\n        try:\n            return
self._process_docling_subprocess_impl(local_path, file_path)\n        finally:\n            #
Clean up temp file if we created one\n            if should_delete:\n                with
contextlib.suppress(Exception):\n                    Path(local_path).unlink()  # Ignore cleanup
errors\n\n    def _process_docling_subprocess_impl(self, local_file_path: str,
original_file_path: str) -> Data | None:\n        \"\"\"Implementation of Docling subprocess
processing.\n\n        Args:\n            local_file_path: Path to local file to process\n
original_file_path: Original file path to include in metadata\n        Returns:\n            Data
object with processed content\n        \"\"\"\n        args: dict[str, Any] = {\n
\"file_path\": local_file_path,\n            \"markdown\": bool(self.markdown),\n
\"image_mode\": str(self.IMAGE_MODE),\n            \"md_image_placeholder\":
str(self.md_image_placeholder),\n            \"md_page_break_placeholder\":
str(self.md_page_break_placeholder),\n            \"pipeline\": str(self.pipeline),\n
\"ocr_engine\": (\n                self.ocr_engine if self.ocr_engine and self.ocr_engine !=
\"None\" and self.pipeline != \"vlm\" else None\n            ),\n        }\n\n        # Child script for
isolating the docling processing\n        child_script = textwrap.dedent(\n            r\"\"\"\n
import json, sys\n\n            def try_imports():\n                try:\n                    from
docling.datamodel.base_models import ConversionStatus, InputFormat  # type:
ignore\n                    from docling.document_converter import DocumentConverter  #
type: ignore\n                    from docling_core.types.doc import ImageRefMode  # type:
ignore\n                    return ConversionStatus, InputFormat, DocumentConverter,
ImageRefMode, \"latest\"\n                except Exception as e:\n                    raise e\n\n
def create_converter(strategy, input_format, DocumentConverter, pipeline,
ocr_engine):\n                # --- Standard PDF/IMAGE pipeline (your existing behavior), with
optional OCR ---\n                if pipeline == \"standard\":\n                    try:\n                        from
docling.datamodel.pipeline_options import PdfPipelineOptions  # type: ignore\n
from docling.document_converter import PdfFormatOption  # type: ignore\n\n
pipe = PdfPipelineOptions()\n                        pipe.do_ocr = False\n\n                        if
ocr_engine:\n                            try:\n                                from docling.models.factories import
```

```
get_ocr_factory  # type: ignore\n                pipe.do_ocr = True\n                fac = get_ocr_factory(allow_external_plugins=False)\n                pipe.ocr_options = fac.create_options(kind=ocr_engine)\n            except Exception:\n                # If OCR setup fails, disable it\n                pipe.do_ocr = False\n\n            fmt = {}\n            if hasattr(input_format, \"PDF\"):\n                fmt[getattr(input_format, \"PDF\")] = PdfFormatOption(pipeline_options=pipe)\n            if hasattr(input_format, \"IMAGE\"):\n                fmt[getattr(input_format, \"IMAGE\")] = PdfFormatOption(pipeline_options=pipe)\n\n            return DocumentConverter(format_options=fmt)\n        except Exception:\n            return DocumentConverter()\n\n    # --- Vision-Language Model (VLM) pipeline ---\n    if pipeline == \"vlm\":\n        try:\n            from docling.datamodel.pipeline_options import VlmPipelineOptions\n            from docling.datamodel.vlm_model_specs import GRANITEDOCLING_MLX, GRANITEDOCLING_TRANSFORMERS\n            from docling.document_converter import PdfFormatOption\n            from docling.pipeline.vlm_pipeline import VlmPipeline\n\n            vl_pipe = VlmPipelineOptions(\n                vlm_options=GRANITEDOCLING_TRANSFORMERS,\n            )\n\n            if sys.platform == \"darwin\":\n                try:\n                    import mlx_vlm\n                    vl_pipe.vlm_options = GRANITEDOCLING_MLX\n                except ImportError as e:\n                    raise e\n\n            # VLM paths generally don't need OCR; keep OCR off by default here.\n            fmt = {}\n            if hasattr(input_format, \"PDF\"):\n                fmt[getattr(input_format, \"PDF\")] = PdfFormatOption(\n                    pipeline_cls=VlmPipeline,\n                    pipeline_options=vl_pipe\n                )\n            if hasattr(input_format, \"IMAGE\"):\n                fmt[getattr(input_format, \"IMAGE\")] = PdfFormatOption(\n                    pipeline_cls=VlmPipeline,\n                    pipeline_options=vl_pipe\n                )\n\n            return DocumentConverter(format_options=fmt)\n        except Exception as e:\n            raise e\n\n    # --- Fallback: default converter with no special options ---\n    return DocumentConverter()\n\n\ndef export_markdown(document, ImageRefMode, image_mode, img_ph, pg_ph):\n    try:\n        mode = getattr(ImageRefMode, image_mode.upper(), image_mode)\n        return document.export_to_markdown(\n            image_mode=mode,\n            image_placeholder=img_ph,\n            page_break_placeholder=pg_ph,\n        )\n    except Exception:\n        try:\n            return document.export_to_text()\n        except Exception:\n            return str(document)\n\n\ndef to_rows(doc_dict):\n    rows = []\n    for t in doc_dict.get(\"texts\", []):\n        prov = t.get(\"prov\") or []\n        page_no = None\n        if prov and isinstance(prov, list) and isinstance(prov[0], dict):\n            page_no = prov[0].get(\"page_no\")\n        rows.append({\n            \"page_no\": page_no,\n            \"label\": t.get(\"label\"),\n            \"text\": t.get(\"text\"),\n            \"level\": t.get(\"level\"),\n        })\n    return rows\n\n\ndef main():\n
```

```
cfg = json.loads(sys.stdin.read())\n            file_path = cfg[\"file_path\"]\n
markdown = cfg[\"markdown\"]\n            image_mode = cfg[\"image_mode\"]\n
img_ph = cfg[\"md_image_placeholder\"]\n            pg_ph =
cfg[\"md_page_break_placeholder\"]\n            pipeline = cfg[\"pipeline\"]\n
ocr_engine = cfg.get(\"ocr_engine\")\n            meta = {\"file_path\": file_path}\n\n
try:\n            ConversionStatus, InputFormat, DocumentConverter, ImageRefMode,
strategy = try_imports()\n            converter = create_converter(strategy, InputFormat,
DocumentConverter, pipeline, ocr_engine)\n            try:\n            res =
converter.convert(file_path)\n            except Exception as e:\n
print(json.dumps({\"ok\": False, \"error\": f\"Docling conversion error: {e}\", \"meta\":
meta}))\n            return\n\n            ok = False\n            if hasattr(res,
\"status\"):\n            try:\n            ok = (res.status ==
ConversionStatus.SUCCESS) or (str(res.status).lower() == \"success\")\n
except Exception:\n            ok = (str(res.status).lower() == \"success\")\n
if not ok and hasattr(res, \"document\"):\n            ok = getattr(res, \"document\",
None) is not None\n            if not ok:\n            print(json.dumps({\"ok\": False,
\"error\": \"Docling conversion failed\", \"meta\": meta}))\n            return\n\n
doc = getattr(res, \"document\", None)\n            if doc is None:\n
print(json.dumps({\"ok\": False, \"error\": \"Docling produced no document\", \"meta\":
meta}))\n            return\n\n            if markdown:\n            text =
export_markdown(doc, ImageRefMode, image_mode, img_ph, pg_ph)\n
print(json.dumps({\"ok\": True, \"mode\": \"markdown\", \"text\": text, \"meta\": meta}))\n
return\n\n            # structured\n            try:\n            doc_dict =
doc.export_to_dict()\n            except Exception as e:\n
print(json.dumps({\"ok\": False, \"error\": f\"Docling export_to_dict failed: {e}\", \"meta\":
meta}))\n            return\n\n            rows = to_rows(doc_dict)\n
print(json.dumps({\"ok\": True, \"mode\": \"structured\", \"doc\": rows, \"meta\":
meta}))\n            except Exception as e:\n            print(\n            json.dumps({\n
\"ok\": False,\n            \"error\": f\"Docling processing error: {e}\",\n
\"meta\": {\"file_path\": file_path},\n            })\n            )\n\n        if __name__ ==
\"__main__\":\n            main()\n        \"\"\"\n    )\n\n    # Validate file_path to avoid
command injection or unsafe input\n    if not isinstance(args[\"file_path\"], str) or
any(c in args[\"file_path\"] for c in [\";\", \"|\", \"&\", \"$\", \"`\"]):\n        return
Data(data={\"error\": \"Unsafe file path detected.\", \"file_path\": args[\"file_path\"]})\n\n
proc = subprocess.run(  # noqa: S603\n        [sys.executable, \"-u\", \"-c\",
child_script],\n        input=json.dumps(args).encode(\"utf-8\"),\n
capture_output=True,\n        check=False,\n    )\n\n    if not proc.stdout:\n
err_msg = proc.stderr.decode(\"utf-8\", errors=\"replace\") if proc.stderr else \"no output
from child process\"\n        return Data(data={\"error\": f\"Docling subprocess error:
{err_msg}\", \"file_path\": original_file_path})\n\n    try:\n        result =
json.loads(proc.stdout.decode(\"utf-8\"))\n    except Exception as e:  # noqa:
```

BLE001\n err_msg = proc.stderr.decode(\"utf-8\", errors=\"replace\")\n return Data(\n data={\n \"error\": f\"Invalid JSON from Docling subprocess: {e}. stderr={err_msg}\",\n \"file_path\": original_file_path,\n },\n)\n\n if not result.get(\"ok\"):\n error_msg = result.get(\"error\", \"Unknown Docling error\")\n # Override meta file_path with original_file_path to ensure correct path matching\n meta = result.get(\"meta\", {})\n meta[\"file_path\"] = original_file_path\n return Data(data={\"error\": error_msg, **meta})\n\n meta = result.get(\"meta\", {})\n # Override meta file_path with original_file_path to ensure correct path matching\n # The subprocess returns the temp file path, but we need the original S3/local path for rollup_data\n meta[\"file_path\"] = original_file_path\n if result.get(\"mode\") == \"markdown\":\n exported_content = str(result.get(\"text\", \"\"))\n return Data(\n text=exported_content,\n data={\"exported_content\": exported_content, \"export_format\": self.EXPORT_FORMAT, **meta},\n)\n\n rows = list(result.get(\"doc\", []))\n return Data(data={\"doc\": rows, \"export_format\": self.EXPORT_FORMAT, **meta})\n\n def process_files(\n self,\n file_list: list[BaseFileComponent.BaseFile],\n) -> list[BaseFileComponent.BaseFile]:\n \"\"\"Process input files.\n\n - advanced_mode => Docling in a separate process.\n - Otherwise => standard parsing in current process (optionally threaded).\n \"\"\"\n if not file_list:\n msg = \"No files to process.\"\n raise ValueError(msg)\n\n # Validate image files to detect content/extension mismatches\n # This prevents API errors like \"Image does not match the provided media type\"\n image_extensions = {\"jpeg\", \"jpg\", \"png\", \"gif\", \"webp\", \"bmp\", \"tiff\"}\n settings = get_settings_service().settings\n for file in file_list:\n extension = file.path.suffix[1:].lower()\n if extension in image_extensions:\n # Read bytes based on storage type\n try:\n if settings.storage_type == \"s3\":\n # For S3 storage, use storage service to read file bytes\n file_path_str = str(file.path)\n content = run_until_complete(read_file_bytes(file_path_str))\n else:\n # For local storage, read bytes directly from filesystem\n content = file.path.read_bytes()\n\n is_valid, error_msg = validate_image_content_type(\n str(file.path),\n content=content,\n)\n if not is_valid:\n self.log(error_msg)\n if not self.silent_errors:\n raise ValueError(error_msg)\n except (OSError, FileNotFoundError) as e:\n self.log(f\"Could not read file for validation: {e}\")\n # Continue - let it fail later with better error\n\n # Validate that files requiring Docling are only processed when advanced mode is enabled\n if not self.advanced_mode:\n for file in file_list:\n extension = file.path.suffix[1:].lower()\n if extension in self.DOCLING_ONLY_EXTENSIONS:\n if is_astra_cloud_environment():\n msg = (\n f\"File '{file.path.name}' has extension '.{extension}' which requires \"\n f\"Advanced Parser mode. Advanced Parser is not available in

```
cloud environments.\"\n                )\n            else:\n                msg = (\n
f\"File '{file.path.name}' has extension '.{extension}' which requires \"\n
f\"Advanced Parser mode. Please enable 'Advanced Parser' to process this file.\"\n
)\n                self.log(msg)\n                raise ValueError(msg)\n\n        def
process_file_standard(file_path: str, *, silent_errors: bool = False) -> Data | None:\n
try:\n            return parse_text_file_to_data(file_path, silent_errors=silent_errors)\n
except FileNotFoundError as e:\n            self.log(f\"File not found: {file_path}. Error:
{e}\")\n            if not silent_errors:\n                raise\n            return None\n        except
Exception as e:\n            self.log(f\"Unexpected error processing {file_path}: {e}\")\n
if not silent_errors:\n                raise\n            return None\n\n        docling_compatible =
all(self._is_docling_compatible(str(f.path)) for f in file_list)\n\n        # Advanced path:
Check if ALL files are compatible with Docling\n        if self.advanced_mode and
docling_compatible:\n            final_return: list[BaseFileComponent.BaseFile] = []\n
for file in file_list:\n                file_path = str(file.path)\n                advanced_data: Data |
None = self._process_docling_in_subprocess(file_path)\n\n                # Handle None case
- Docling processing failed or returned None\n                if advanced_data is None:\n
error_data = Data(\n                    data={\n                        \"file_path\": file_path,\n
\"error\": \"Docling processing returned no result. Check logs for details.\",\n
},\n                )\n                final_return.extend(self.rollup_data([file], [error_data]))\n
continue\n\n                # --- UNNEST: expand each element in `doc` to its own Data row\n
payload = getattr(advanced_data, \"data\", {}) or {}\n\n                # Check for errors first\n
if \"error\" in payload:\n                    error_msg = payload.get(\"error\", \"Unknown error\")\n
error_data = Data(\n                    data={\n                        \"file_path\": file_path,\n
\"error\": error_msg,\n                        **{k: v for k, v in payload.items() if k not in (\"error\",
\"file_path\")},\n                    },\n                )\n
final_return.extend(self.rollup_data([file], [error_data]))\n                    continue\n\n
doc_rows = payload.get(\"doc\")\n                if isinstance(doc_rows, list) and doc_rows:\n
# Non-empty list of structured rows\n                    rows: list[Data | None] = [\n
Data(\n                            data={\n                                \"file_path\": file_path,\n
**(item if isinstance(item, dict) else {\"value\": item}),\n                            },\n                        )\n
for item in doc_rows\n                    ]\n                    final_return.extend(self.rollup_data([file],
rows))\n                elif isinstance(doc_rows, list) and not doc_rows:\n                    # Empty list
- file was processed but no text content found\n                    # Create a Data object
indicating no content was extracted\n                    self.log(f\"No text extracted from
'{file_path}', creating placeholder data\")\n                    empty_data = Data(\n
data={\n                            \"file_path\": file_path,\n                            \"text\": \"(No text content
extracted from image)\",\n                            \"info\": \"Image processed successfully but
contained no extractable text\",\n                            **{k: v for k, v in payload.items() if k !=
\"doc\"},\n                        },\n                    )\n
final_return.extend(self.rollup_data([file], [empty_data]))\n                else:\n                    # If
not structured, keep as-is (e.g., markdown export or error dict)\n                    # Ensure
```

file_path is set for proper rollup matching\n if not payload.get(\"file_path\"):\n payload[\"file_path\"] = file_path\n # Create new Data with file_path\n advanced_data = Data(\n data=payload,\n text=getattr(advanced_data, \"text\", None),\n)\n final_return.extend(self.rollup_data([file], [advanced_data]))\n return final_return\n\n # Standard multi-file (or single non-advanced) path\n concurrency = 1 if not self.use_multithreading else max(1, self.concurrency_multithreading)\n\n file_paths = [str(f.path) for f in file_list]\n self.log(f\"Starting parallel processing of {len(file_paths)} files with concurrency: {concurrency}.\")\n my_data = parallel_load_data(\n file_paths,\n silent_errors=self.silent_errors,\n load_function=process_file_standard,\n max_concurrency=concurrency,\n)\n return self.rollup_data(file_list, my_data)\n\n # ------------------------------ Output helpers ------------------------------------\n\n def load_files_helper(self) -> DataFrame:\n result = self.load_files()\n\n # Result is a DataFrame - check if it has any rows\n if result.empty:\n msg = \"Could not extract content from the provided file(s).\"\n raise ValueError(msg)\n\n # Check for error column with error messages\n if \"error\" in result.columns:\n errors = result[\"error\"].dropna().tolist()\n if errors and not any(col in result.columns for col in [\"text\", \"doc\", \"exported_content\"]):\n raise ValueError(errors[0])\n\n return result\n\n def load_files_dataframe(self) -> DataFrame:\n \"\"\"Load files using advanced Docling processing and export to DataFrame format.\"\"\"\n self.markdown = False\n return self.load_files_helper()\n\n def load_files_markdown(self) -> Message:\n \"\"\"Load files using advanced Docling processing and export to Markdown format.\"\"\"\n self.markdown = True\n result = self.load_files_helper()\n\n # Result is a DataFrame - check for text or exported_content columns\n if \"text\" in result.columns and not result[\"text\"].isna().all():\n text_values = result[\"text\"].dropna().tolist()\n if text_values:\n return Message(text=str(text_values[0]))\n\n if \"exported_content\" in result.columns and not result[\"exported_content\"].isna().all():\n content_values = result[\"exported_content\"].dropna().tolist()\n if content_values:\n return Message(text=str(content_values[0]))\n\n # Return empty message with info that no text was found\n return Message(text=\"(No text content extracted from file)\")\n"

},

"concurrency_multithreading": {

"_input_type": "IntInput",

"advanced": true,

"display_name": "Processing Concurrency",

```
        "dynamic": false,
        "info": "When multiple files are being processed, the number of files to process concurrently.",
        "list": false,
        "list_add_label": "Add More",
        "name": "concurrency_multithreading",
        "override_skip": false,
        "placeholder": "",
        "required": false,
        "show": true,
        "title_case": false,
        "tool_mode": false,
        "trace_as_metadata": true,
        "track_in_telemetry": true,
        "type": "int",
        "value": 1
      },
      "delete_server_file_after_processing": {
        "_input_type": "BoolInput",
        "advanced": true,
        "display_name": "Delete Server File After Processing",
        "dynamic": false,
        "info": "If true, the Server File Path will be deleted after processing.",
        "list": false,
        "list_add_label": "Add More",
        "name": "delete_server_file_after_processing",
        "override_skip": false,
        "placeholder": "",
```

```
  "required": false,
  "show": true,
  "title_case": false,
  "tool_mode": false,
  "trace_as_metadata": true,
  "track_in_telemetry": true,
  "type": "bool",
  "value": true
 },
 "doc_key": {
  "_input_type": "MessageTextInput",
  "advanced": true,
  "display_name": "Doc Key",
  "dynamic": false,
  "info": "The key to use for the DoclingDocument column.",
  "input_types": [
   "Message"
  ],
  "list": false,
  "list_add_label": "Add More",
  "load_from_db": false,
  "name": "doc_key",
  "override_skip": false,
  "placeholder": "",
  "required": false,
  "show": false,
  "title_case": false,
  "tool_mode": false,
```

```
          "trace_as_input": true,
          "trace_as_metadata": true,
          "track_in_telemetry": false,
          "type": "str",
          "value": "doc"
        },
        "file_id": {
          "_input_type": "StrInput",
          "advanced": false,
          "display_name": "Google Drive File ID",
          "dynamic": false,
          "info": "The Google Drive file ID to read. The file must be shared with the service account email.",
          "list": false,
          "list_add_label": "Add More",
          "load_from_db": false,
          "name": "file_id",
          "override_skip": false,
          "placeholder": "",
          "required": true,
          "show": false,
          "title_case": false,
          "tool_mode": false,
          "trace_as_metadata": true,
          "track_in_telemetry": false,
          "type": "str",
          "value": ""
        },
```

```
"file_path": {
 "_input_type": "HandleInput",
 "advanced": true,
 "display_name": "Server File Path",
 "dynamic": false,
 "info": "Data object with a 'file_path' property pointing to server file or a Message object with a path to the file. Supercedes 'Path' but supports same file types.",
 "input_types": [
  "Data",
  "Message"
 ],
 "list": true,
 "list_add_label": "Add More",
 "name": "file_path",
 "override_skip": false,
 "placeholder": "",
 "required": false,
 "show": true,
 "title_case": false,
 "trace_as_metadata": true,
 "track_in_telemetry": false,
 "type": "other",
 "value": ""
},
"file_path_str": {
 "_input_type": "StrInput",
 "advanced": true,
 "display_name": "File Path",
```

```
"dynamic": false,
"info": "Path to the file to read. Used when component is called as a tool. If not provided, will use the uploaded file from 'path' input.",
"list": false,
"list_add_label": "Add More",
"load_from_db": false,
"name": "file_path_str",
"override_skip": false,
"placeholder": "",
"required": false,
"show": false,
"title_case": false,
"tool_mode": true,
"trace_as_metadata": true,
"track_in_telemetry": false,
"type": "str",
"value": ""
},
"ignore_unspecified_files": {
"_input_type": "BoolInput",
"advanced": true,
"display_name": "Ignore Unspecified Files",
"dynamic": false,
"info": "If true, Data with no 'file_path' property will be ignored.",
"list": false,
"list_add_label": "Add More",
"name": "ignore_unspecified_files",
"override_skip": false,
```

```
  "placeholder": "",
  "required": false,
  "show": true,
  "title_case": false,
  "tool_mode": false,
  "trace_as_metadata": true,
  "track_in_telemetry": true,
  "type": "bool",
  "value": false
 },
 "ignore_unsupported_extensions": {
  "_input_type": "BoolInput",
  "advanced": true,
  "display_name": "Ignore Unsupported Extensions",
  "dynamic": false,
  "info": "If true, files with unsupported extensions will not be processed.",
  "list": false,
  "list_add_label": "Add More",
  "name": "ignore_unsupported_extensions",
  "override_skip": false,
  "placeholder": "",
  "required": false,
  "show": true,
  "title_case": false,
  "tool_mode": false,
  "trace_as_metadata": true,
  "track_in_telemetry": true,
  "type": "bool",
```

```
      "value": true
    },
    "is_refresh": false,
    "markdown": {
      "_input_type": "BoolInput",
      "advanced": false,
      "display_name": "Markdown Export",
      "dynamic": false,
      "info": "Export processed documents to Markdown format. Only available when advanced mode is enabled.",
      "list": false,
      "list_add_label": "Add More",
      "name": "markdown",
      "override_skip": false,
      "placeholder": "",
      "required": false,
      "show": false,
      "title_case": false,
      "tool_mode": false,
      "trace_as_metadata": true,
      "track_in_telemetry": true,
      "type": "bool",
      "value": false
    },
    "md_image_placeholder": {
      "_input_type": "StrInput",
      "advanced": true,
      "display_name": "Image placeholder",
```

```
  "dynamic": false,
  "info": "Specify the image placeholder for markdown exports.",
  "list": false,
  "list_add_label": "Add More",
  "load_from_db": false,
  "name": "md_image_placeholder",
  "override_skip": false,
  "placeholder": "",
  "required": false,
  "show": false,
  "title_case": false,
  "tool_mode": false,
  "trace_as_metadata": true,
  "track_in_telemetry": false,
  "type": "str",
  "value": "<!-- image -->"
 },
 "md_page_break_placeholder": {
  "_input_type": "StrInput",
  "advanced": true,
  "display_name": "Page break placeholder",
  "dynamic": false,
  "info": "Add this placeholder between pages in the markdown output.",
  "list": false,
  "list_add_label": "Add More",
  "load_from_db": false,
  "name": "md_page_break_placeholder",
  "override_skip": false,
```

```
  "placeholder": "",
  "required": false,
  "show": false,
  "title_case": false,
  "tool_mode": false,
  "trace_as_metadata": true,
  "track_in_telemetry": false,
  "type": "str",
  "value": ""
 },
 "ocr_engine": {
  "_input_type": "DropdownInput",
  "advanced": true,
  "combobox": false,
  "dialog_inputs": {},
  "display_name": "OCR Engine",
  "dynamic": false,
  "external_options": {},
  "info": "OCR engine to use. Only available when pipeline is set to 'standard'.",
  "name": "ocr_engine",
  "options": [
   "None",
   "easyocr"
  ],
  "options_metadata": [],
  "override_skip": false,
  "placeholder": "",
  "required": false,
```

```
  "show": false,
  "title_case": false,
  "toggle": false,
  "tool_mode": false,
  "trace_as_metadata": true,
  "track_in_telemetry": true,
  "type": "str",
  "value": "easyocr"
},
"path": {
  "_input_type": "FileInput",
  "advanced": false,
  "display_name": "Files",
  "dynamic": false,
  "fileTypes": [
    "csv",
    "json",
    "pdf",
    "txt",
    "md",
    "mdx",
    "yaml",
    "yml",
    "xml",
    "html",
    "htm",
    "docx",
    "py",
```

"sh",

"sql",

"js",

"ts",

"tsx",

"adoc",

"asciidoc",

"asc",

"bmp",

"dotx",

"dotm",

"docm",

"jpg",

"jpeg",

"png",

"potx",

"ppsx",

"pptm",

"potm",

"ppsm",

"pptx",

"tiff",

"xls",

"xlsx",

"xhtml",

"webp",

"zip",

"tar",

```
          "tgz",
          "bz2",
          "gz"
        ],
        "file_path": [
          "46b74e69-899d-4849-83ba-1d9a32ea1981/### Test Cases for Doctor Prescript (1).txt"
        ],
        "info": "Supported file extensions: csv, json, pdf, txt, md, mdx, yaml, yml, xml, html, htm, docx, py, sh, sql, js, ts, tsx, adoc, asciidoc, asc, bmp, dotx, dotm, docm, jpg, jpeg, png, potx, ppsx, pptm, potm, ppsm, pptx, tiff, xls, xlsx, xhtml, webp; optionally bundled in file extensions: zip, tar, tgz, bz2, gz",
        "list": true,
        "list_add_label": "Add More",
        "name": "path",
        "override_skip": false,
        "placeholder": "",
        "real_time_refresh": true,
        "required": false,
        "show": true,
        "temp_file": false,
        "title_case": false,
        "tool_mode": false,
        "trace_as_metadata": true,
        "track_in_telemetry": false,
        "type": "file",
        "value": ""
      },
      "pipeline": {
```

```
  "_input_type": "DropdownInput",
  "advanced": true,
  "combobox": false,
  "dialog_inputs": {},
  "display_name": "Pipeline",
  "dynamic": false,
  "external_options": {},
  "info": "Docling pipeline to use",
  "name": "pipeline",
  "options": [
    "standard",
    "vlm"
  ],
  "options_metadata": [],
  "override_skip": false,
  "placeholder": "",
  "real_time_refresh": true,
  "required": false,
  "show": true,
  "title_case": false,
  "toggle": false,
  "tool_mode": false,
  "trace_as_metadata": true,
  "track_in_telemetry": true,
  "type": "str",
  "value": "standard"
},
"s3_file_key": {
```

```
            "_input_type": "StrInput",
            "advanced": false,
            "display_name": "S3 File Key",
            "dynamic": false,
            "info": "The key (path) of the file in S3 bucket.",
            "list": false,
            "list_add_label": "Add More",
            "load_from_db": false,
            "name": "s3_file_key",
            "override_skip": false,
            "placeholder": "",
            "required": true,
            "show": false,
            "title_case": false,
            "tool_mode": false,
            "trace_as_metadata": true,
            "track_in_telemetry": false,
            "type": "str",
            "value": ""
          },
          "separator": {
            "_input_type": "StrInput",
            "advanced": true,
            "display_name": "Separator",
            "dynamic": false,
            "info": "Specify the separator to use between multiple outputs in Message format.",
            "list": false,
```

```
            "list_add_label": "Add More",
            "load_from_db": false,
            "name": "separator",
            "override_skip": false,
            "placeholder": "",
            "required": false,
            "show": true,
            "title_case": false,
            "tool_mode": false,
            "trace_as_metadata": true,
            "track_in_telemetry": false,
            "type": "str",
            "value": "\n\n"
          },
          "service_account_key": {
            "_input_type": "SecretStrInput",
            "advanced": false,
            "display_name": "GCP Credentials Secret Key",
            "dynamic": false,
            "info": "Your Google Cloud Platform service account JSON key as a secret string (complete JSON content).",
            "input_types": [],
            "load_from_db": false,
            "name": "service_account_key",
            "override_skip": false,
            "password": true,
            "placeholder": "",
            "required": true,
```

```
  "show": false,
  "title_case": false,
  "track_in_telemetry": false,
  "type": "str",
  "value": ""
 },
 "silent_errors": {
  "_input_type": "BoolInput",
  "advanced": true,
  "display_name": "Silent Errors",
  "dynamic": false,
  "info": "If true, errors will not raise an exception.",
  "list": false,
  "list_add_label": "Add More",
  "name": "silent_errors",
  "override_skip": false,
  "placeholder": "",
  "required": false,
  "show": true,
  "title_case": false,
  "tool_mode": false,
  "trace_as_metadata": true,
  "track_in_telemetry": true,
  "type": "bool",
  "value": false
 },
 "storage_location": {
  "_input_type": "SortableListInput",
```

```
"advanced": true,
"display_name": "Storage Location",
"dynamic": false,
"info": "Choose where to read the file from.",
"limit": 1,
"name": "storage_location",
"options": [
 {
  "icon": "hard-drive",
  "name": "Local"
 },
 {
  "icon": "Amazon",
  "name": "AWS"
 },
 {
  "icon": "google",
  "name": "Google Drive"
 }
],
"override_skip": false,
"placeholder": "Select Location",
"real_time_refresh": true,
"required": false,
"search_category": [],
"show": true,
"title_case": false,
"tool_mode": false,
```

```
  "trace_as_metadata": true,
  "track_in_telemetry": false,
  "type": "sortableList",
  "value": [
   {
    "chosen": false,
    "icon": "hard-drive",
    "name": "Local",
    "selected": false
   }
  ]
 },
 "use_multithreading": {
  "_input_type": "BoolInput",
  "advanced": true,
  "display_name": "[Deprecated] Use Multithreading",
  "dynamic": false,
  "info": "Set 'Processing Concurrency' greater than 1 to enable multithreading.",
  "list": false,
  "list_add_label": "Add More",
  "name": "use_multithreading",
  "override_skip": false,
  "placeholder": "",
  "required": false,
  "show": true,
  "title_case": false,
  "tool_mode": false,
  "trace_as_metadata": true,
```

```
        "track_in_telemetry": true,
        "type": "bool",
        "value": true
       }
     },
     "tool_mode": false
   },
   "selected_output": "message",
   "showNode": true,
   "type": "File"
  },
  "id": "File-uI9B7",
  "measured": {
   "height": 262,
   "width": 320
  },
  "position": {
   "x": -916.6816713652947,
   "y": 215.27777084500795
  },
  "selected": false,
  "type": "genericNode"
 },
 {
  "data": {
   "id": "TextOutput-gW8O8",
   "node": {
    "base_classes": [
```

```
  "Message"
],
"beta": false,
"conditional_paths": [],
"custom_fields": {},
"description": "Sends text output via API.",
"display_name": "Text Output",
"documentation": "https://docs.langflow.org/text-input-and-output",
"edited": false,
"field_order": [
  "input_value"
],
"frozen": false,
"icon": "type",
"legacy": false,
"lf_version": "1.8.2",
"metadata": {
  "code_hash": "6aba888f4632",
  "dependencies": {
    "dependencies": [
      {
        "name": "lfx",
        "version": null
      }
    ],
    "total_dependencies": 1
  },
  "module": "lfx.components.input_output.text_output.TextOutputComponent"
```

```
},
"minimized": false,
"output_types": [],
"outputs": [
 {
  "allows_loop": false,
  "cache": true,
  "display_name": "Output Text",
  "group_outputs": false,
  "method": "text_response",
  "name": "text",
  "selected": "Message",
  "tool_mode": true,
  "types": [
   "Message"
  ],
  "value": "__UNDEFINED__"
 }
],
"pinned": false,
"template": {
 "_type": "Component",
 "code": {
  "advanced": true,
  "dynamic": true,
  "fileTypes": [],
  "file_path": "",
  "info": "",
```

```
"list": false,
"load_from_db": false,
"multiline": true,
"name": "code",
"password": false,
"placeholder": "",
"required": true,
"show": true,
"title_case": false,
"type": "code",
"value": "from lfx.base.io.text import TextComponent\nfrom lfx.io import MultilineInput, Output\nfrom lfx.schema.message import Message\n\n\nclass TextOutputComponent(TextComponent):\n    display_name = \"Text Output\"\n    description = \"Sends text output via API.\"\n    documentation: str = \"https://docs.langflow.org/text-input-and-output\"\n    icon = \"type\"\n    name = \"TextOutput\"\n\n    inputs = [\n        MultilineInput(\n            name=\"input_value\",\n            display_name=\"Inputs\",\n            info=\"Text to be passed as output.\",\n        ),\n    ]\n    outputs = [\n        Output(display_name=\"Output Text\", name=\"text\", method=\"text_response\"),\n    ]\n\n    def text_response(self) -> Message:\n        message = Message(\n            text=self.input_value,\n        )\n        self.status = self.input_value\n        return message\n"
},
"input_value": {
  "_input_type": "MultilineInput",
  "advanced": false,
  "ai_enabled": false,
  "copy_field": false,
  "display_name": "Inputs",
  "dynamic": false,
  "info": "Text to be passed as output.",
  "input_types": [
```

```
        "Message"
      ],
      "list": false,
      "list_add_label": "Add More",
      "load_from_db": false,
      "multiline": true,
      "name": "input_value",
      "override_skip": false,
      "password": false,
      "placeholder": "",
      "required": false,
      "show": true,
      "title_case": false,
      "tool_mode": false,
      "trace_as_input": true,
      "trace_as_metadata": true,
      "track_in_telemetry": false,
      "type": "str",
      "value": ""
    }
   },
   "tool_mode": false
  },
  "showNode": true,
  "type": "TextOutput"
},
"dragging": false,
"id": "TextOutput-gW8O8",
```

```
      "measured": {
        "height": 206,
        "width": 320
      },
      "position": {
        "x": 319.2760514360742,
        "y": 272.76745587988825
      },
      "selected": true,
      "type": "genericNode"
    }
  ],
  "viewport": {
    "x": 554.079283962967,
    "y": 119.54654475231447,
    "zoom": 0.5970815935372955
  }
 },
 "description": "Empowering Enterprises with Intelligent Interactions.",
 "endpoint_name": null,
 "id": "efca400a-280a-4dfb-887a-fd5eac1eaa2b",
 "is_component": false,
 "last_tested_version": "1.8.2",
 "locked": false,
 "name": "Test case generator for health care",
 "tags": []
}
```